System Design

Basics

- Keep web tier stateless
- Build redundancy at every tier
- Cache data as much as you can
- Support multiple data centers
- Host static assets in CDN
- Scale your data tier by sharding
    - Sharding actually favors denormalization because running join queries across multiple shards is expensive
- Split tiers into individual services
- Monitor your system and use automation tools

Back of envelop estimations

- Power of 2 table

Power	Short Name	Approx Value

10	1 KB	1 thousand
20	1 MB	1 million
30	1 GB	1 billion
40	1 TB	1 trillion

- Important latency numbers

Order of time	Activities
nanosecond	• L1 / L2 cache reference • Branch mispredict • Mutex lock / unlock • Main memory reference
microsecond	• Compress 1 KB with zippy • send 2 KB over 1 GBPS network • Sequential read 1 MB from memory • Round trip within same DC
millisecond	• Disk seek • Sequential read 1 MB from network • Sequential read 1 MB from disk • Send packet CA → Dutch → CA

- Availability table

Availability %	Downtime per year
99.9 %	3.65 days
99.99 %	52.6 hours
99.999 %	5.26 minutes

Final tips and conclusions

- Memory is fast but the disk is slow.
- Avoid disk seeks if possible.
- Simple compression algorithms are fast.

- Compress data before sending it over the internet if possible.
- Data centers are usually in different regions, and it takes time to send data between them.
- Rounding and approximation are fine, precision is not expected.
- Write down your assumptions & label your units.
- Prepare for QPS, peak QPS, storage, cache & number of servers.

Framework for interviews

- STEP 1 — 10 minutes : Gather scope requirements, public vs private service ? estimate back of envelope estimations ... gather important features that MUST be implemented, chose a design pattern that matches and scales with your estimates ... question the nature and order of data hitting your ingress and egress endpoints ... vertical vs horizontal scalability ... underlying networking nuances
- STEP 2 — 10 minutes : propose high level design and get buy-in, follow KISS
- STEP 3 — 30 minutes : go deep on critical components of your design ... ask relevant questions, stress-test your assumptions ... think about CAP, ACID and SOLID principles ... its a conversation, not a monologue ... go over the trade-offs
- STEP 4 — Wrap up — 10 mins : talk about testing frameworks, compatibility of this design with the existing environment, ask for actual implementations already running at the company and gauge the gaps between your design and the actual design

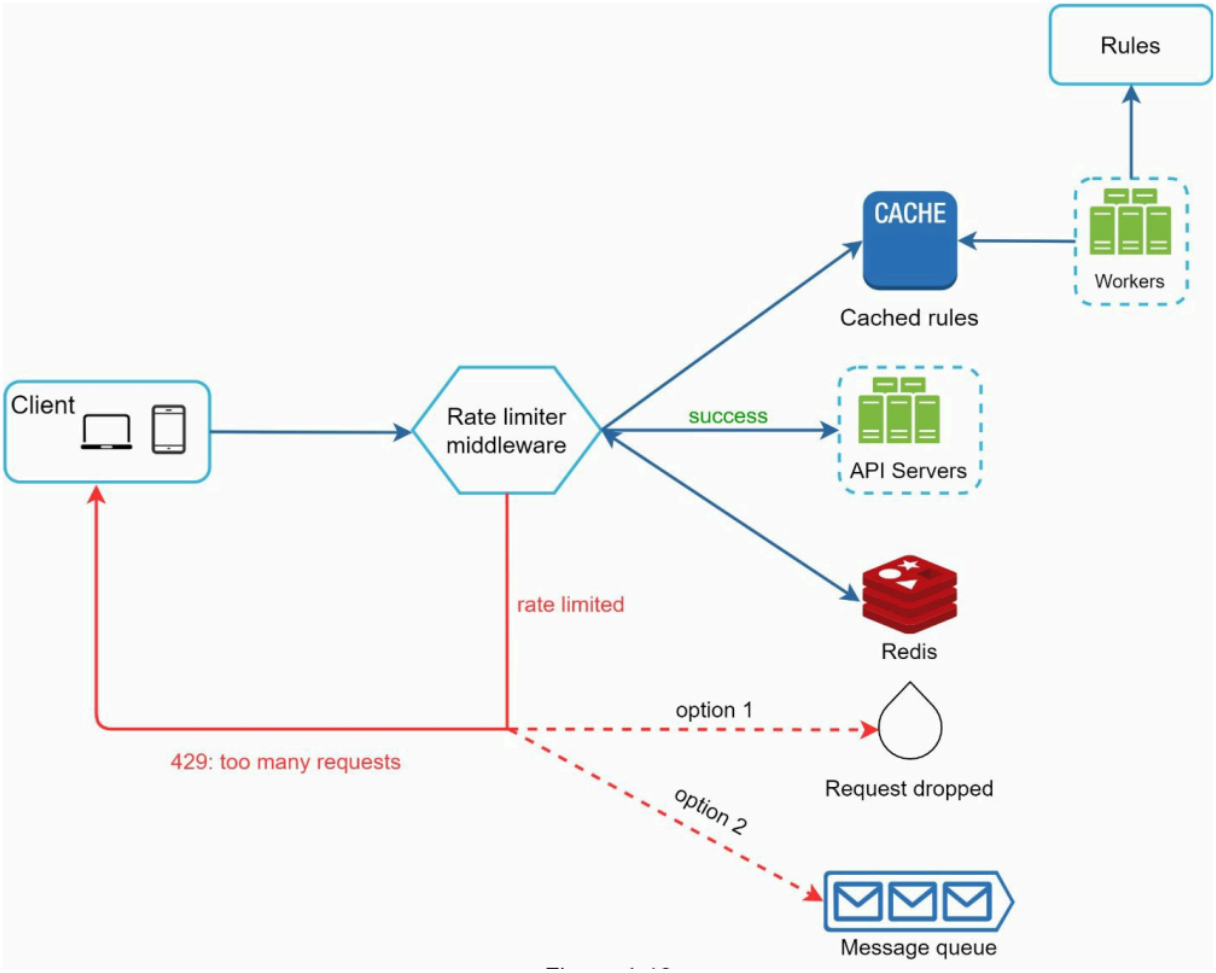
Commentaries on caching

- Types — server side, client side, CDN caching
- Expiration — designed on the basis of data patterns and back of the envelope estimations

- Eviction policies — also dependent on data patterns, LRU and and LFU being the most common
- Write policies — write through (update both - cache & storage), write back (update cache, storage later), write around (bypass the storage entirely)

Rate Limiter

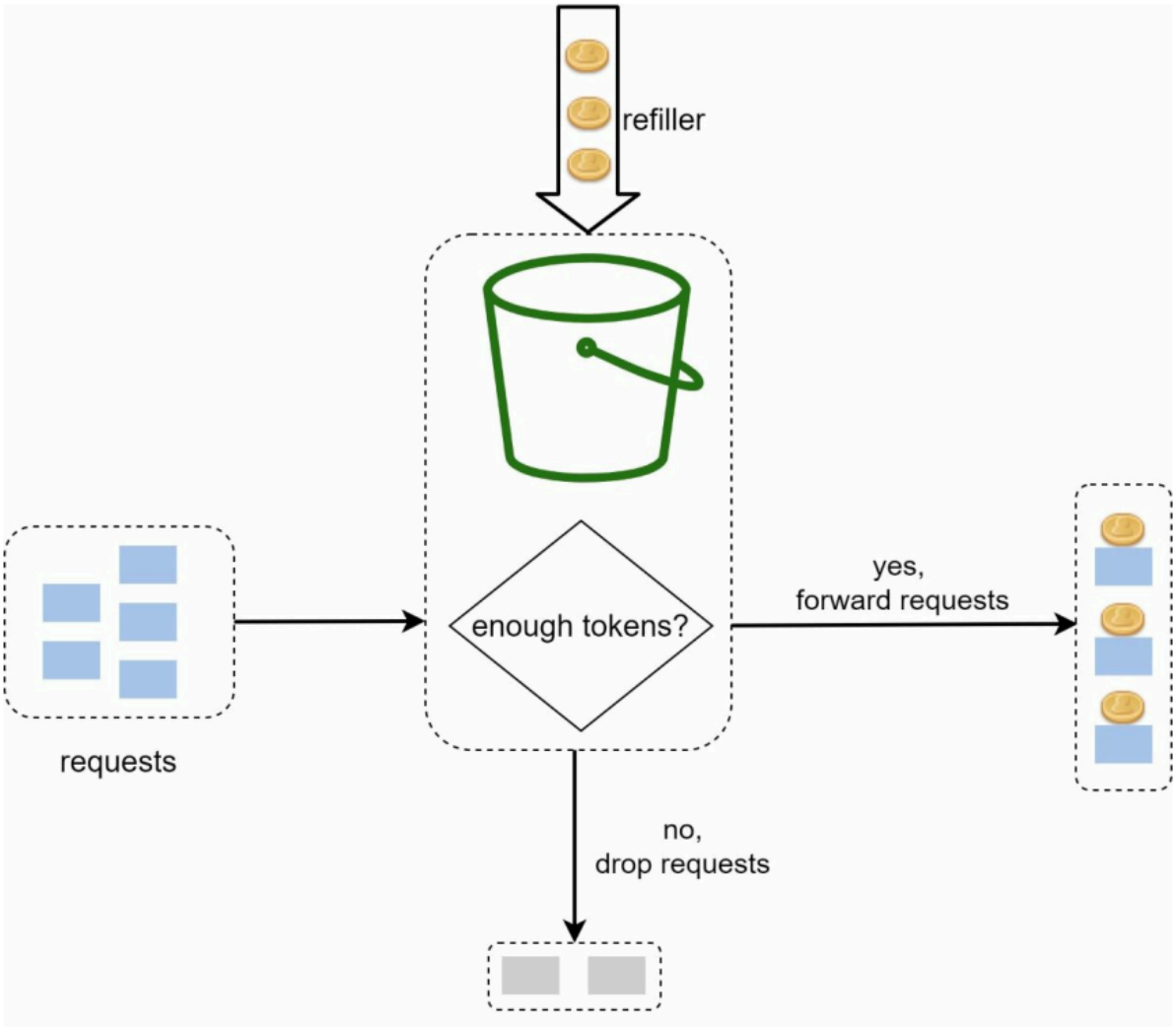
- why rate limiter ? prevents resource starvation (DDOS), reduces 3rd party API cost, reduce server overload
- Types (server, client, middle-ware & API gateway) — implementation depends on CAP trade-offs and data patterns — user throttle notification preferred — client side rate limiting can be spoofed due to lack of control so generally not preferred



Rate limiting algorithms

Token bucket

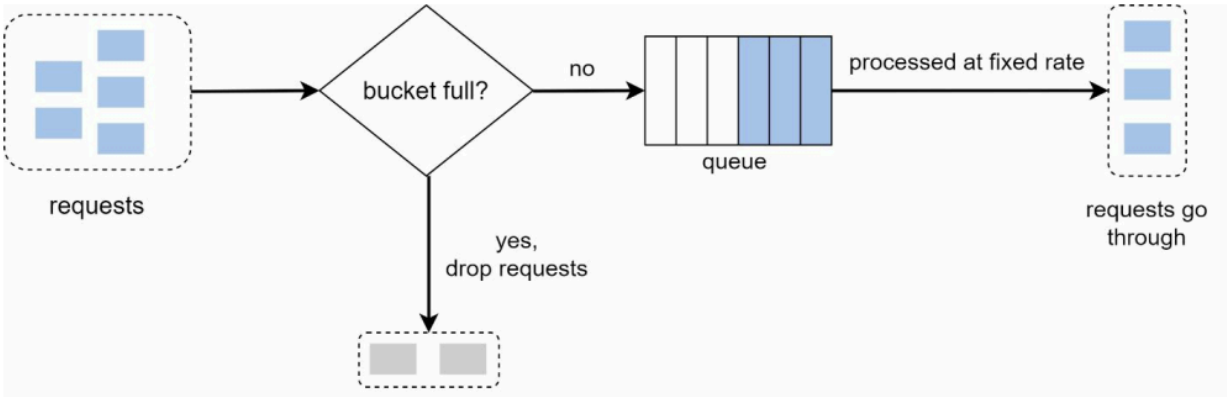
- 2 params (bucket size - max token capacity per bucket, refill rate - number of tokens added per second)
- number of buckets per limiting features (IP address, API endpoint)



Leaking bucket

- essentially token bucket with FIFO

- 2 params (bucket/queue size - max capacity & outflow rate - requests popped from queue per second)

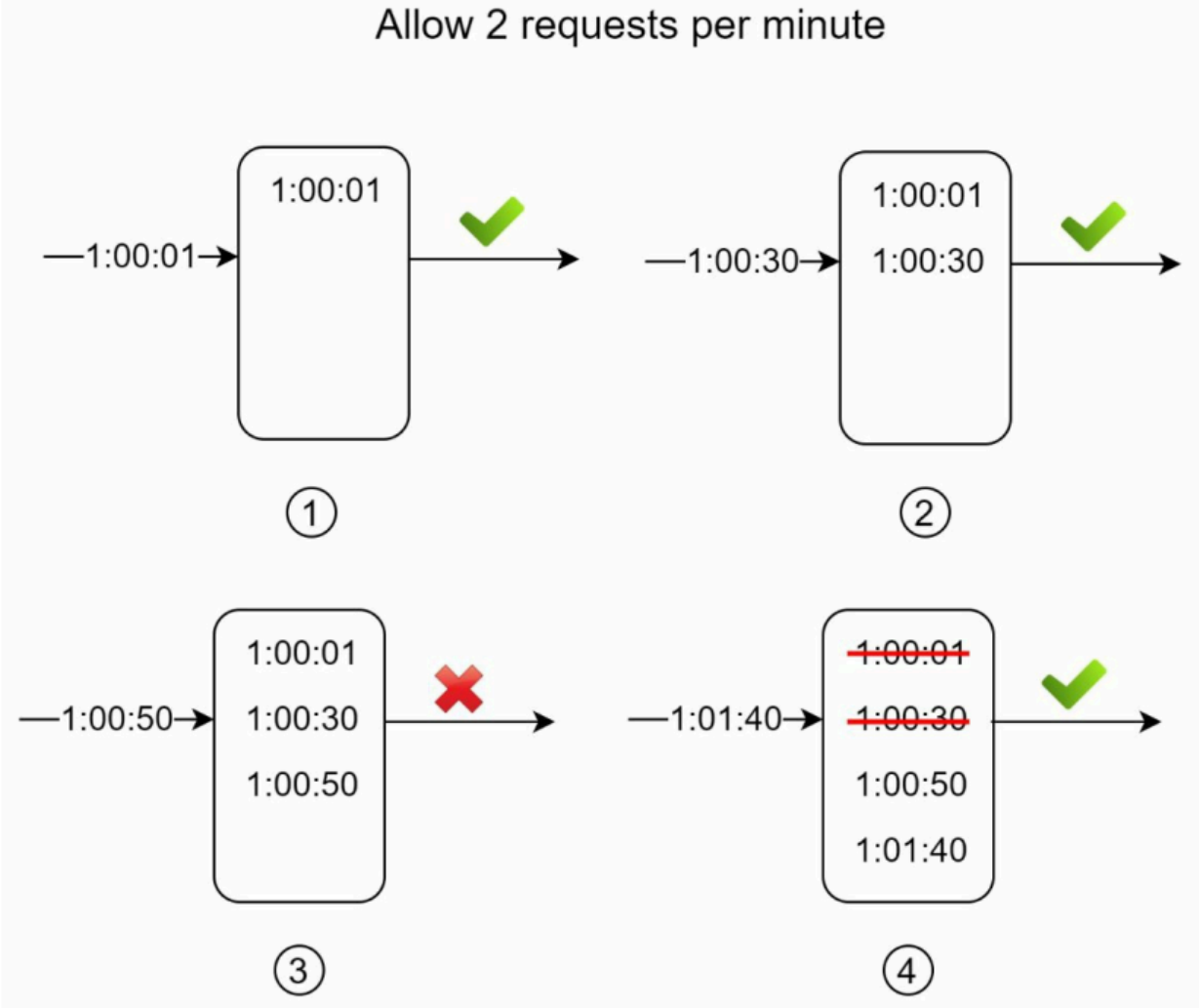


Fixed window counter

- fixed request capacity per timeline slices
- 2 params (window_size - in seconds & request_capacity - number allowed per window)

Sliding window log

- tracks request timestamps
- 2 params (log_size & max_requests)
- old timestamps (before current window start time) are flushed out with each new entry being added to the log
- requests dropped when log_size exceeds max_requests — dropped request timestamp is also added to the log



Sliding window counter

```
Requests in current window + requests in the previous window * overlap percentage of the rolling window and previous window
```

Other optimizations

- Counters for rate limiters are stored using an in-memory cache .. DB would be too slow
- Prefer to enqueue the rate-limited requests to be processed later
- Rate limiter headers : -
  - X-Ratelimit-Remaining: remaining number of allowed requests within the window

    - X-Ratelimit-Limit: indicates how many calls the client can make per time window
    - X-Ratelimit-Retry-After: seconds to wait until you can make a request again
- Distributed rate limiter challenges: -
    - Race condition
        - If two requests concurrently read the counter value before either of them writes the value back, each will increment the counter by one and write it back without checking the other thread
        - Locks are a terrible resolution idea because they are slow
    - Synchronization issue
        - sticky sessions are a cheap remedy here because of poor scaling dynamics

Consistent hashing

- Consistent hashing is an effective technique to mitigate a cache miss storm when one of the cache node crashes — rehashing changes the hash of all keys if the number of servers are affected, consistent hashing only affects the keys stored on impacted server

SHA-1 is used as the hash function f, and the output range of the hash function is: *x0*, *x1*, *x2*, *x3*, …, *xn*. In cryptography, SHA-1's hash space goes from 0 to 2^160 - 1. That means *x0* corresponds to 0, *xn* corresponds to 2^160 – 1, and all the other hash values in the middle fall between 0 and 2^160 - 1. Figure 5-3 shows the hash space.

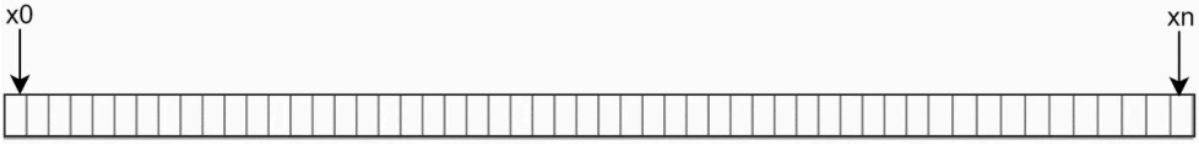
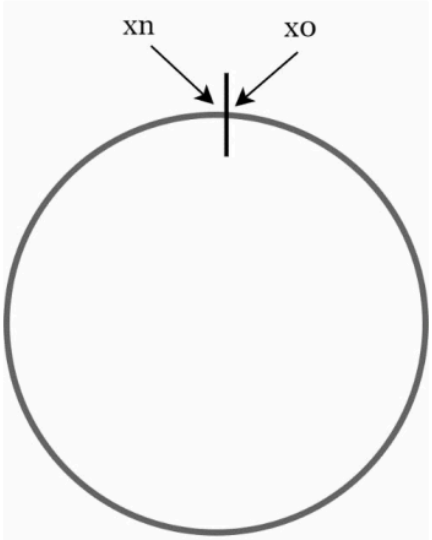
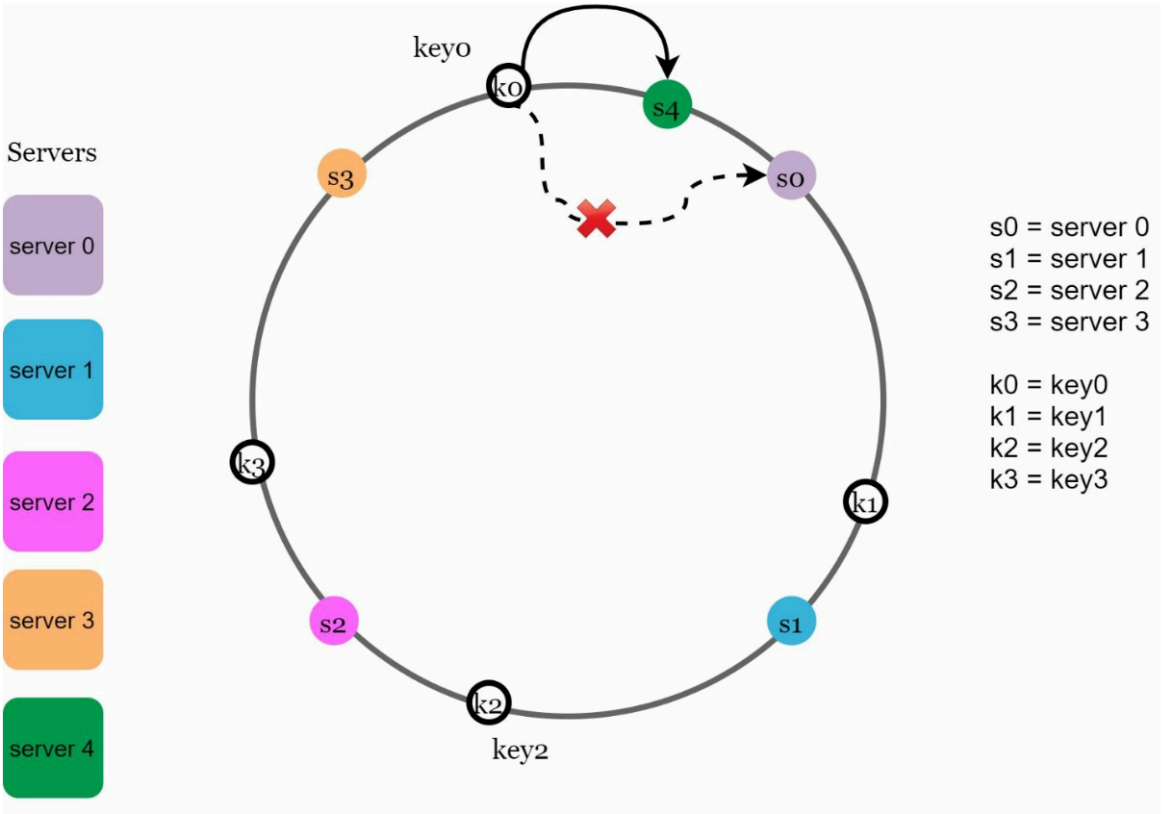


Figure 5-3

By collecting both ends, we get a hash ring as shown in Figure 5-4:



- `Hash ring` is usually traversed clockwise
- Partitioning is used to decrease standard deviation between multiple servers … increase in number of partitions reduces standard deviation at the cost of storage size required to store server_partition information … partitions are effectively virtual nodes on a server
- what happens to a set of keys when s0 is removed and s4 is added as replacement



- partitioned servers resulting in low standard deviation — preferred to keep partitions from different servers in between each other

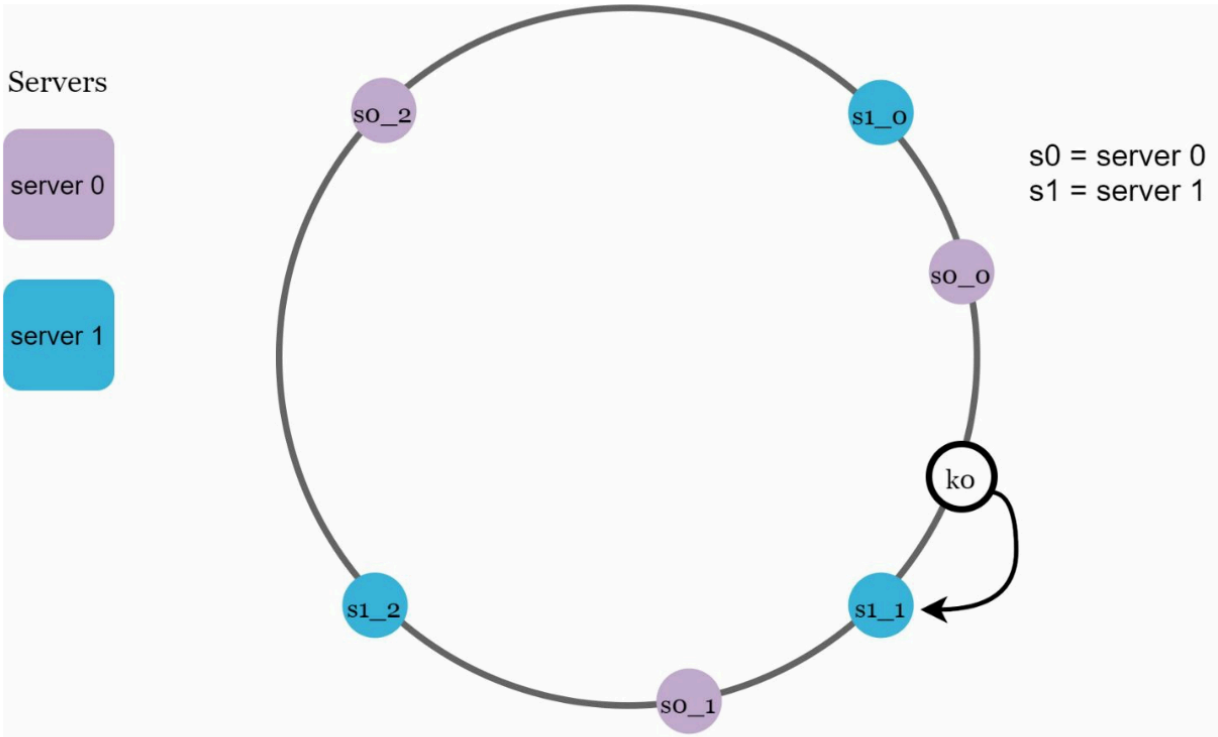


Figure 5-13

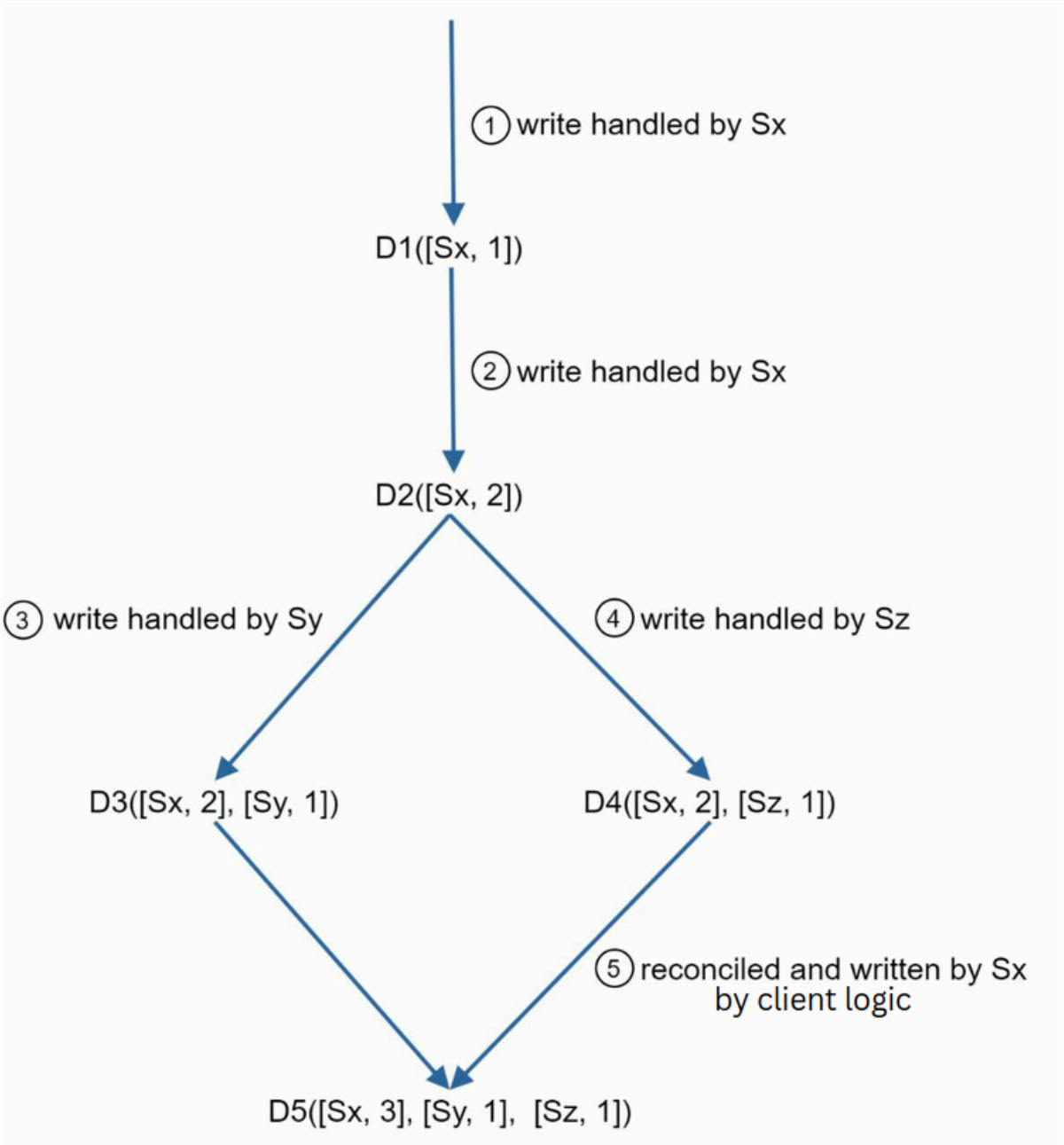
Key Value store

- Design mainly revolves around C(consistency)A(availability)P(partition tolerance) trade-offs ... a distributed system can only satisfy 2 of these attributes simultaneously
    - `CA system isn't possible in the real world` because it sacrifices partition tolerance, in the real world a network failure isn't 100% preventable so partition tolerance is pretty much mandatory

Consistency

(Write quorum, Read quorum, Number of partitions) : different data partitions must reflect the same value using asynchronous replication, guaranteed by quorum consistency

- Strong consistency — any read operation returns a value corresponding to the result of the most updated write data item. A client never sees out-of-date data. W+R > N
- Weak consistency — subsequent read operations may not see the most updated value
- Eventual consistency — this is a specific form of weak consistency. Given enough time, all updates are propagated, and all replicas are consistent. W+R < N
- Inconsistency resolution through versioning : -
  - requires client intervention in case of conflict
  - A `vector clock` is used to keep track of latest versions of data-points written/modified by each server
  - Cons of a vector clock : client conflict resolution mechanism required; server:version pairs grow rapidly and a length limit might be required where older versions are discarded



Failure detection

- All to all multicast is very inefficient, `gossip protocol` is better — each node maintains a node membership list, each node sends periodic heartbeats to a random set of nodes which in turn propagates to other set of random nodes.
- If the heartbeat has not increased for more than predefined periods, the member is

considered as offline.

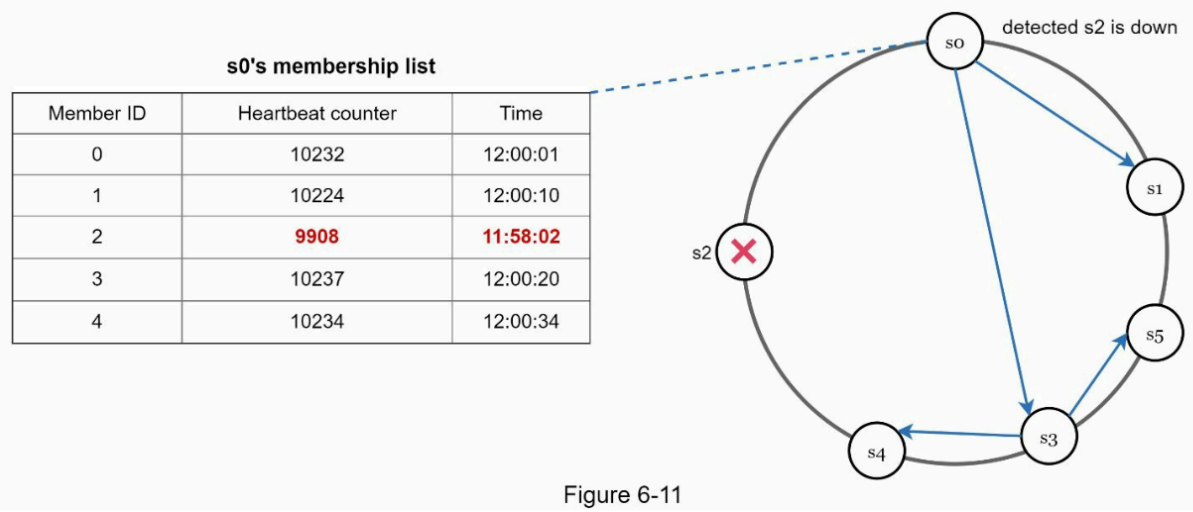


Figure 6-11

- Handling temporary failure
  - `Sloppy quorum` with `hinted-hand-off` mechanism is used to ensure availability when one of the nodes is unavailable for a short period of time
  - Sloppy quorum : instead of enforcing the quorum requirement, the system chooses the first W healthy servers for writes and first R healthy servers for reads on the hash ring
  - Hinted hand-off : If a server is unavailable due to network or server failures, another server will process requests temporarily. When the down server is up, changes will be pushed back to achieve data consistency
- Handling permanent failure
  - We need anti-entropy algorithms that involve comparing each piece of data on replicas and updating each replica to the newest version. A Merkle tree is used for inconsistency detection and minimizing the amount of data transferred
  - A `hash tree or Merkle tree` is a tree in which every non-leaf node is labeled with the hash of the labels or values (in case of leaves) of its child nodes. Hash trees allow efficient and secure verification of the contents of large data structures
- Final system architecture diagram

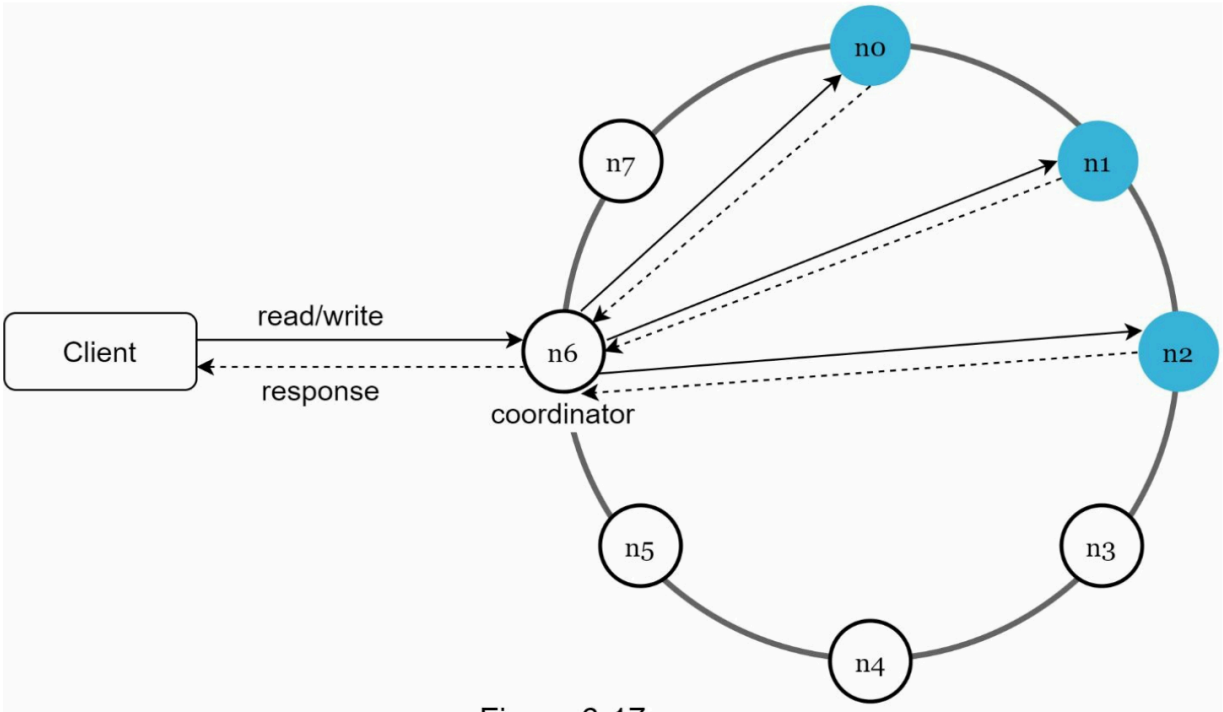
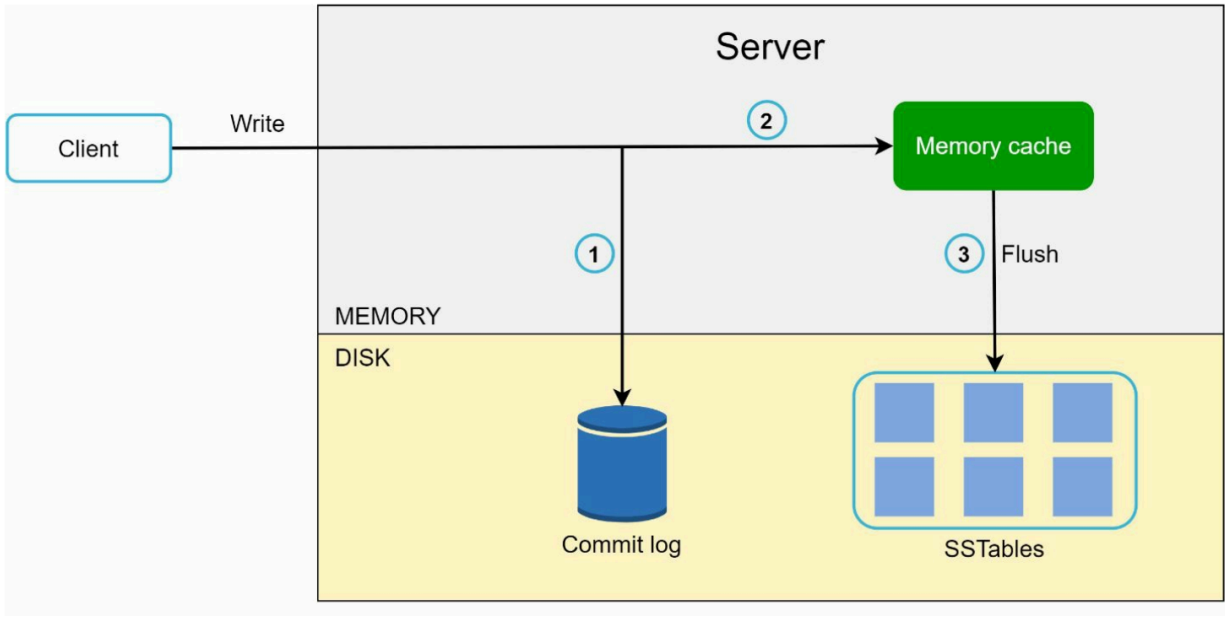
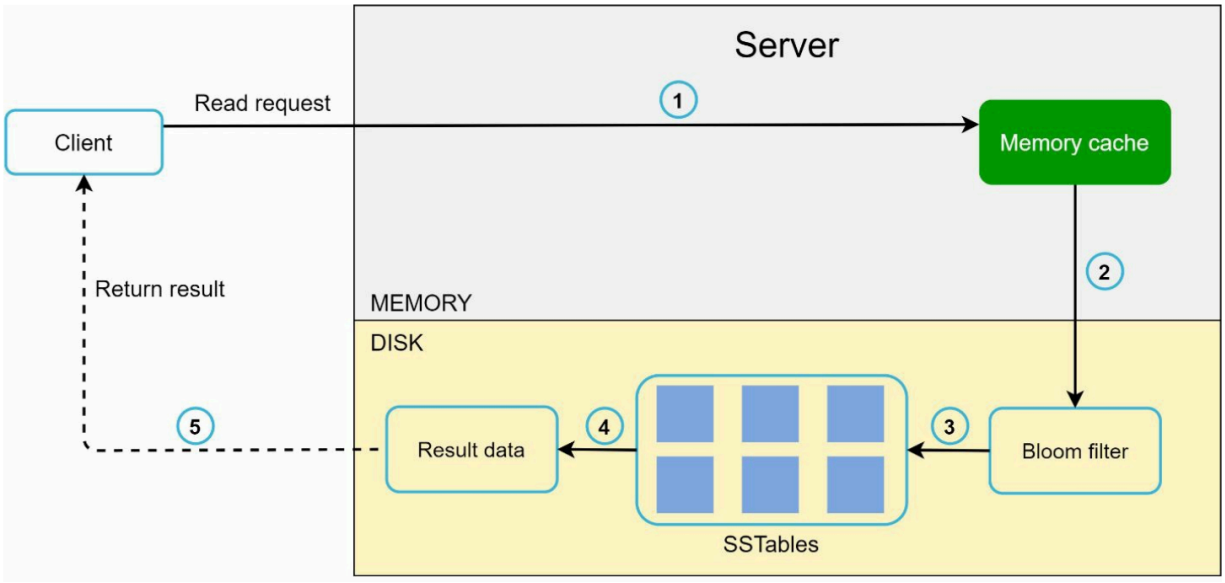


Figure 6-17

- Write path



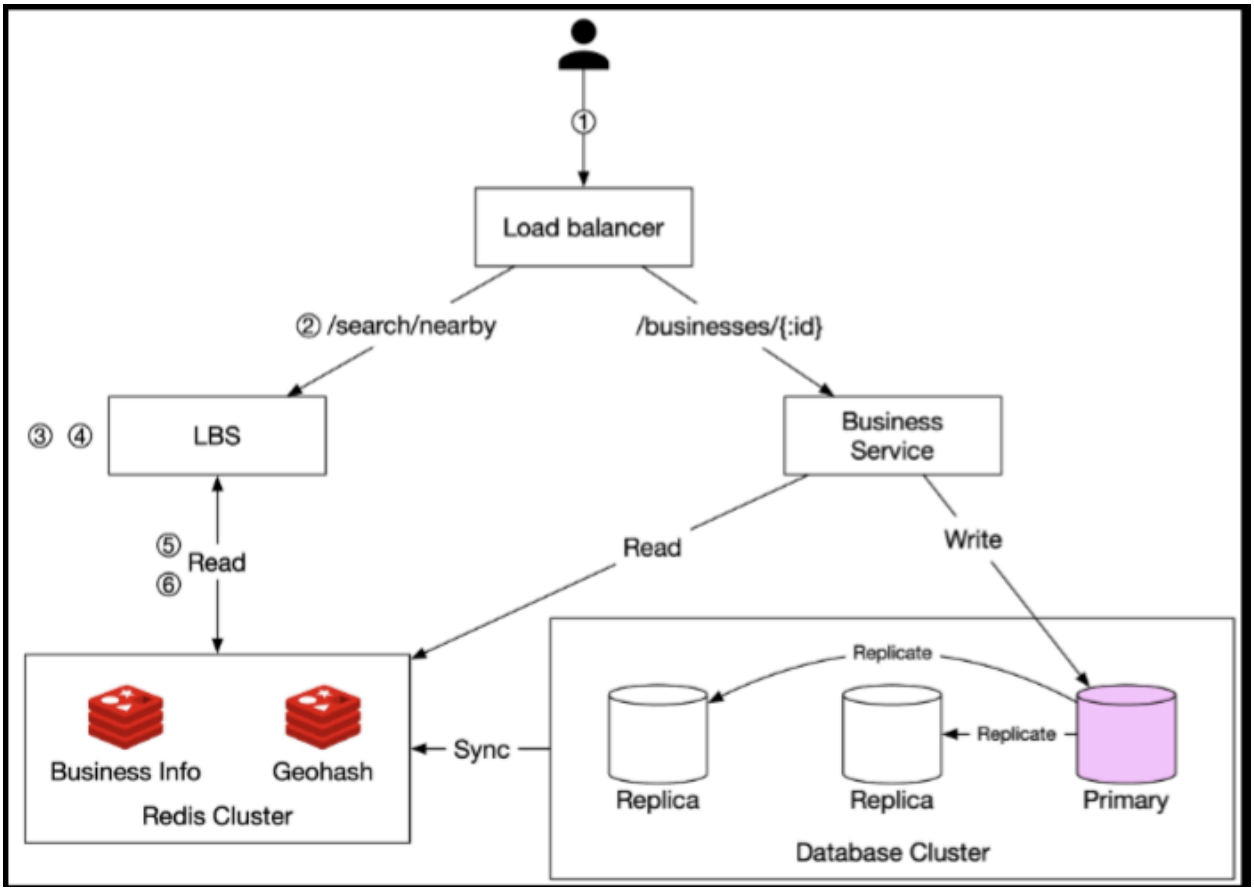
- Read path



Proximity Service

Base functionality includes : -

- Return all businesses based on a user's location and radius
- Business owners can add, delete or update a business, but this doesn't have to be reflected in real time
- Customers can view detailed information about a business



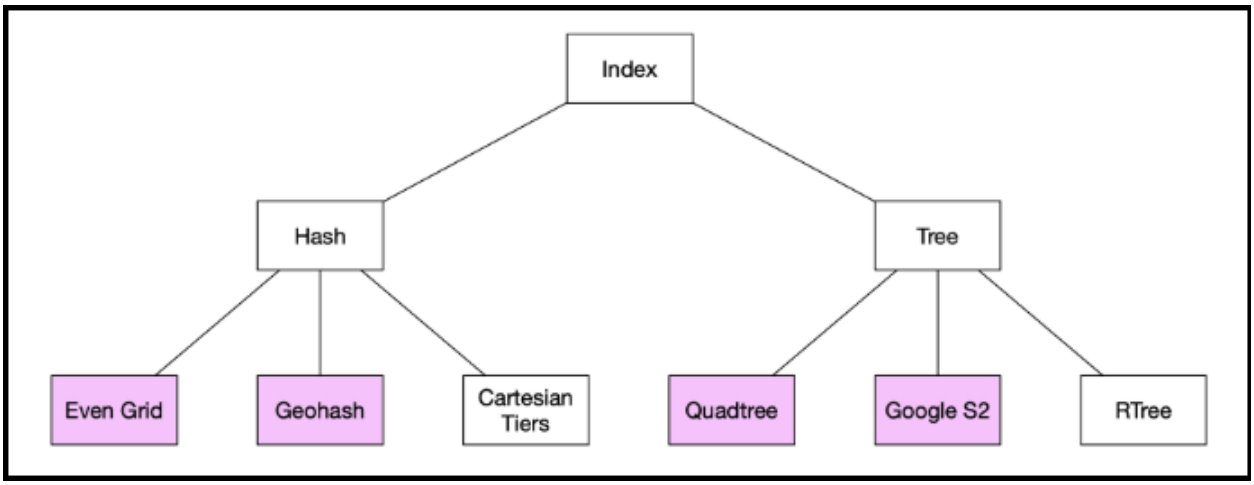
Location based service

- Read heavy service used to serve geospatial data about nearby businesses

Business service

- Allows business owners to update business details
- Also allows customers to view details about businesses

Geospatial data requires special indexing techniques to reduce search latency in 2 dimensional data



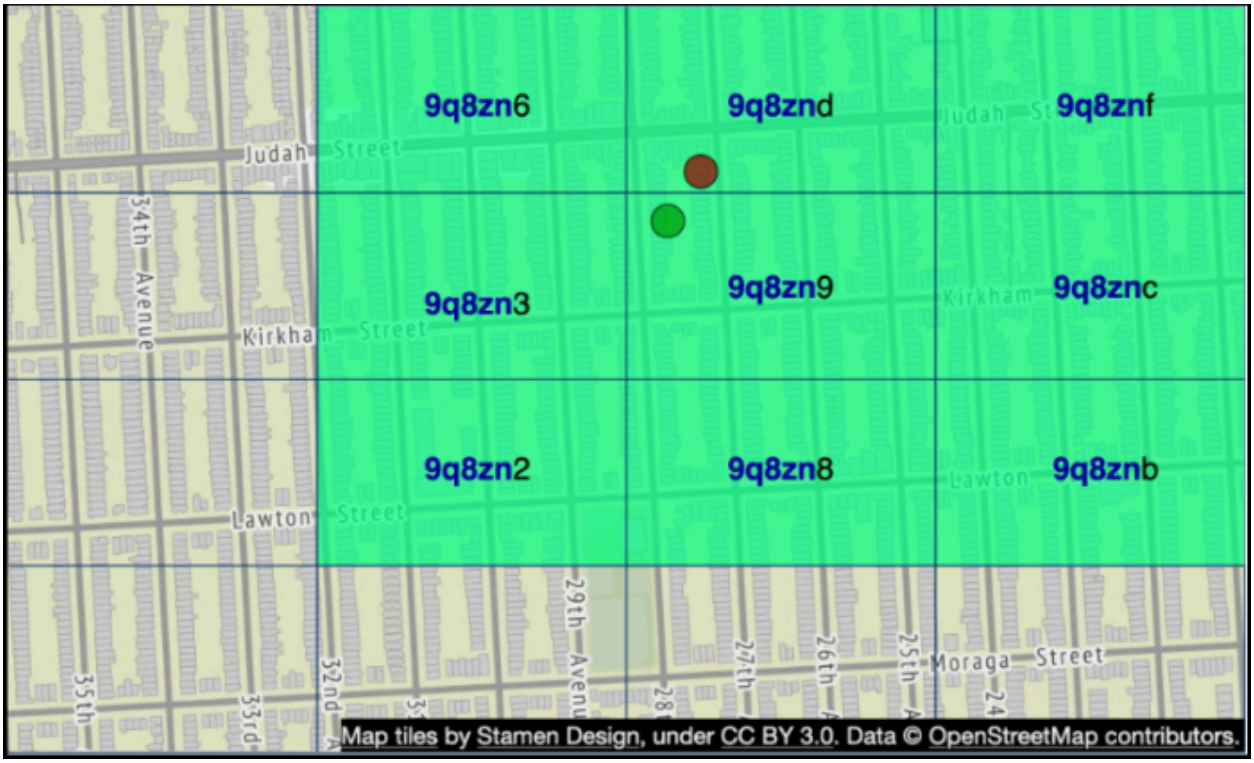
Geohash

- The idea is to break down 2 dimensional data into one dimensional base32 represented strings
- Woks by breaking down the planet into smaller grids recursively
- Geohash has 12 precision leaves, refer table below

Geohash length	Grid width x height
1	5,009.4km x 4,992.6km (the size of the planet)
2	1,252.3km x 624.1km
3	156.5km x 156km
4	39.1km x 19.5km
5	4.9km x 4.9km
6	1.2km x 609.4m
7	152.9m x 152.4m
8	38.2m x 19m
9	4.8m x 4.8m
10	1.2m x 59.5cm
11	14.9cm x 14.9cm
12	3.7cm x 1.9cm

Geohash boundary problems

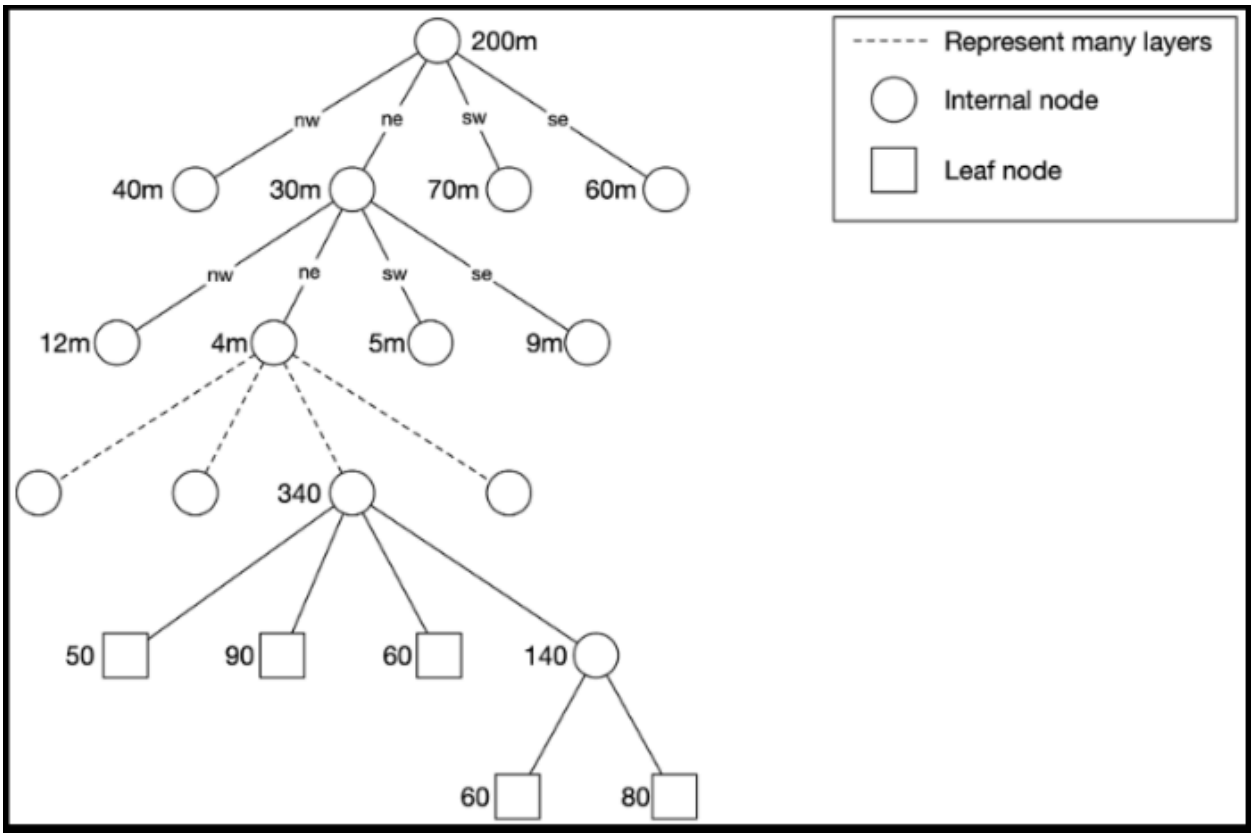
- 2 locations can be very close but have no shared prefix at all, the vice-versa is not true ... if 2 locations have the same prefix then they are definitely close — prefix SQL queries are useless
- 2 locations can have a long shared prefix but they belong to different geohashes, you need a diagram to understand this



- Geohash can help you in situations where there are not enough businesses in a geohash and its neighbors ... you simply remove the last character of the string and return results from a larger hash

Quadtree

- This is an in-memory data structure and does not store data in a DB
- Instead of encoded hashes, we build a tree where each child is recursively smaller region of its parent until we hit a predefined number of businesses in a region



- The amount of memory required for a tree is actually not so high, for 200 million businesses (remember, we are only storing the index, not all the data in memory)
  - memory required = 2 GB
  - construction time required = 2 minutes
- Blue-Green deployment is necessary because a server can't stream traffic while building a new version of the tree
- Quadtree would require a new build to register a new business, it would also result in cache invalidation for multiple keys at the same time — so a brown-out situation is very difficult to avoid in real life

Google S2

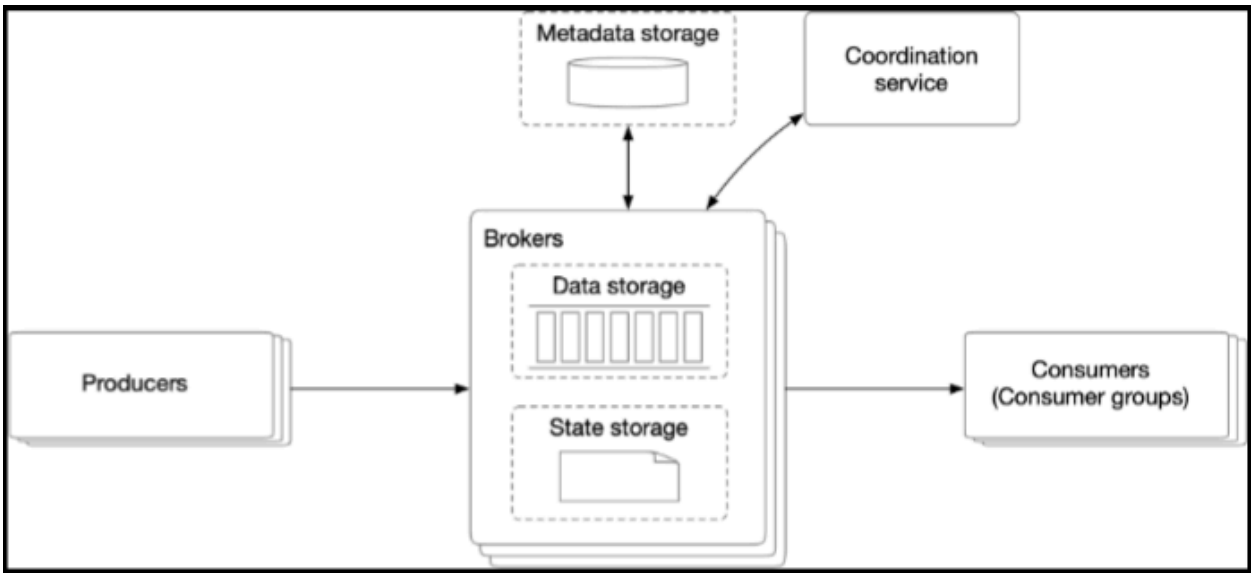
- Crazy math library to map a sphere to a 1 dimensional index based on Hilbert curve

    - Hilbert curve ensures that 2 points closer to each other on the Hilbert curve are also closer to each other in 1 dimensional space — plus, searching in 1D is more efficient than 2D
- Widely used for geofencing
- S2 also supports region cover algorithms with input params like min_level, max_level instead of a predefined precision like geohash

Scaling the storage

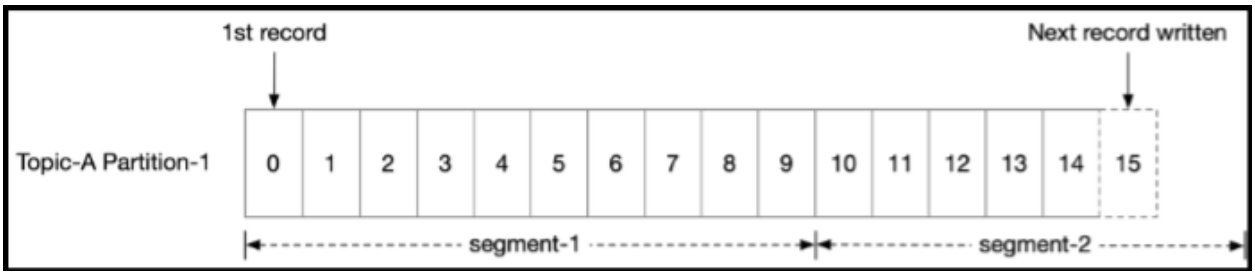
- Business table : sharding based on business ID
- Geospatial index table : no need of sharding because we are dealing with 2 GB of memory which can easily fit in most modern servers, just add read replicas to handle throughput and availability
- Caching can be used BUT it is very tricky with spatial data — best cache keys are business IDs and geohash key — you are better off caching UI presentation data and serving live data from the backend

Distributed Message Queue (my niche)



WAL

- WAL has a pure sequential read/write access pattern
- A new message is appended to the tail of a partition, with a monotonically increasing offset. The easiest option is to use the line number of the log file as the offset. However, a file cannot grow infinitely, so it is a good idea to divide it into segments.
- With segments, new messages are appended only to the active segment file. When the active segment reaches a certain size, a new active segment is created to receive new messages, and the currently active segment becomes inactive, like the rest of the non-active segments.
- Non-active segments only serve read requests. Old non-active segment files can be truncated if they exceed the retention or capacity limit.



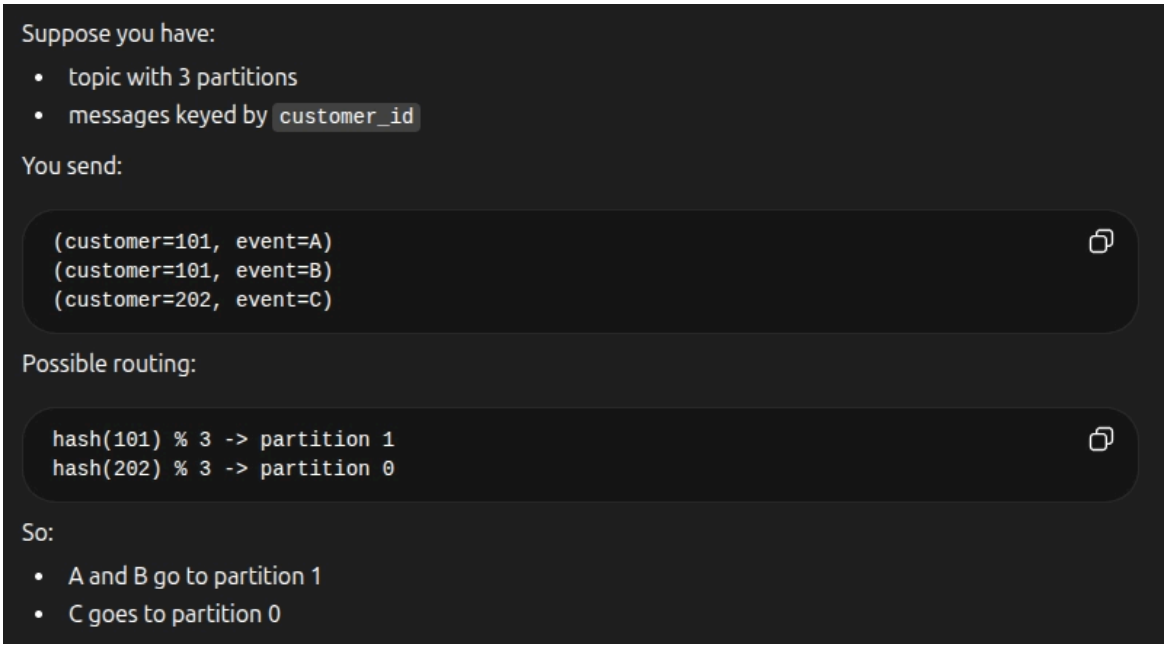
Message Data Structure

```
{
key : byte[],
value : byte[],
topic : string,
partition : integer,
offset : long,
timestamp : long,
size : integer,
crc : integer
}
```

- Message key — this is a business information key mainly used by clients, this doesn't directly map to the partition number, if partition number is absent then

a random partition is used ... if the key is present then the default mapping assigns partition based on `key % numPartitions`

- Message value — payload
- topic — name of topic the message belongs to
- partition — ID of partition the message belongs to
- offset — position of message in the partition
- timestamp — well, timestamp when the message is recorded in the system
- size — size of message
- crc — cyclic redundancy check used to ensure integrity of raw data

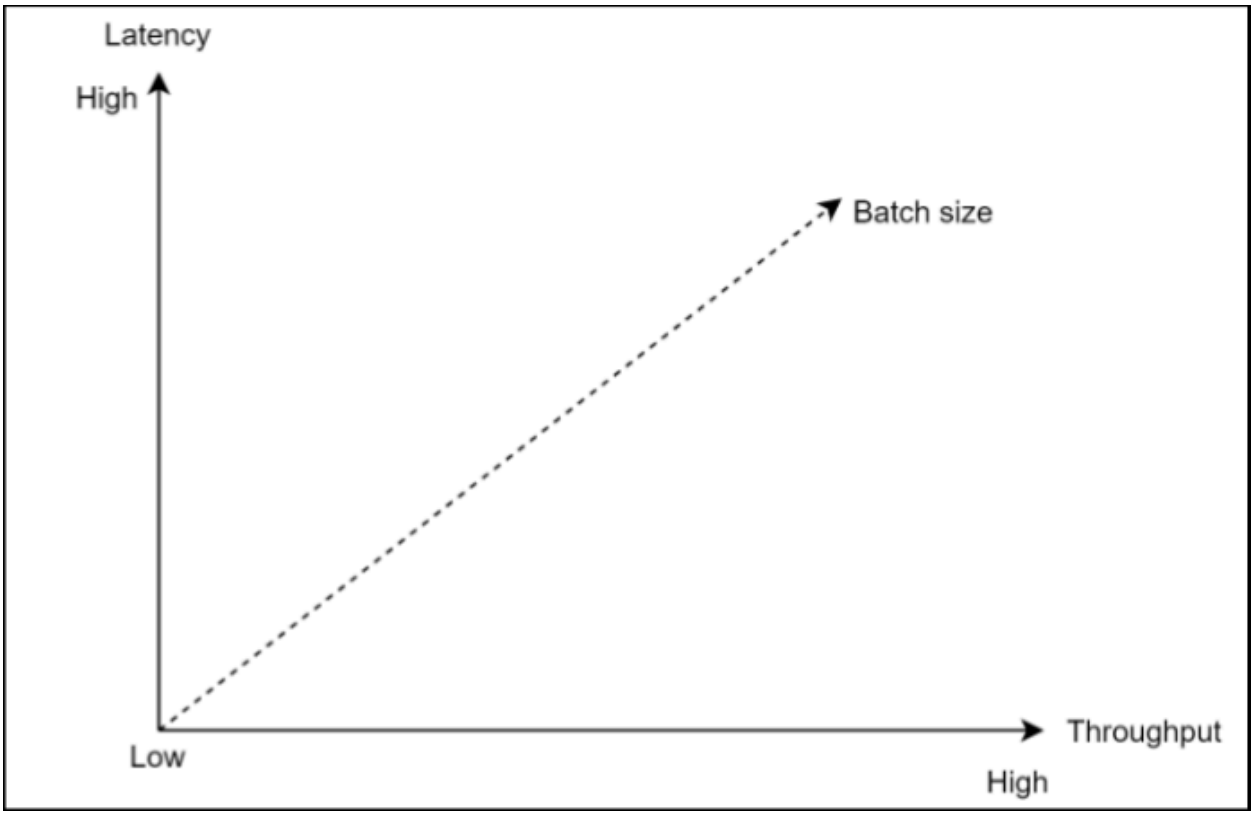


Batching

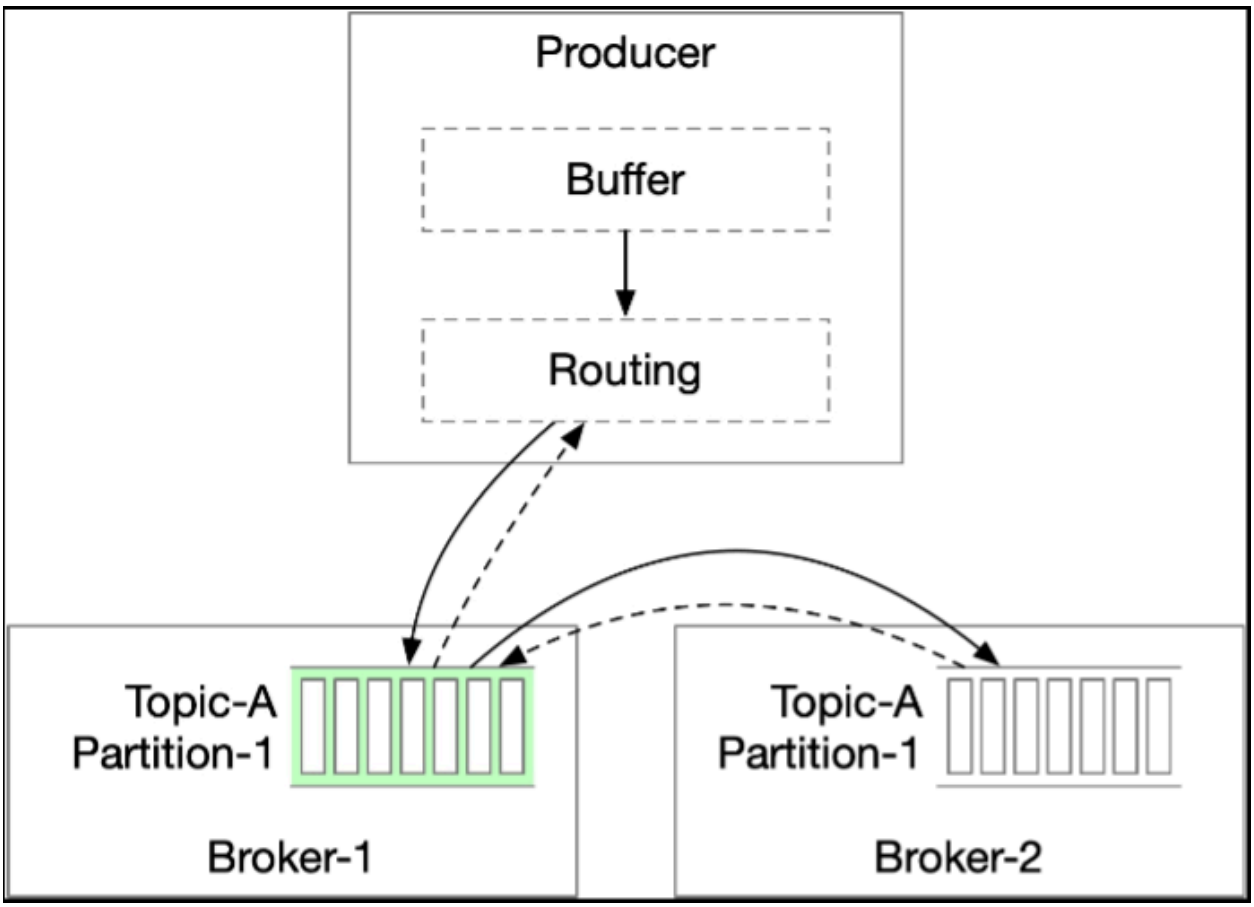
Important for performance because : -

- Allows the operating system to group messages together in a single network request and amortizes the cost of expensive network round trips
- Broker writes messages to append logs in large chunks, which leads to larger blocks of sequential writes and larger contiguous blocks of disk cache maintained by the OS

Both lead to much greater sequential disk access throughput

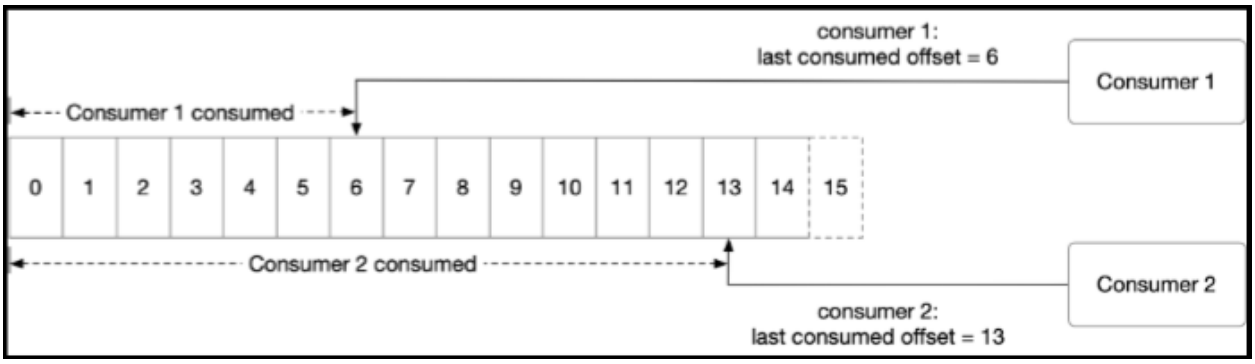


Producer Flow



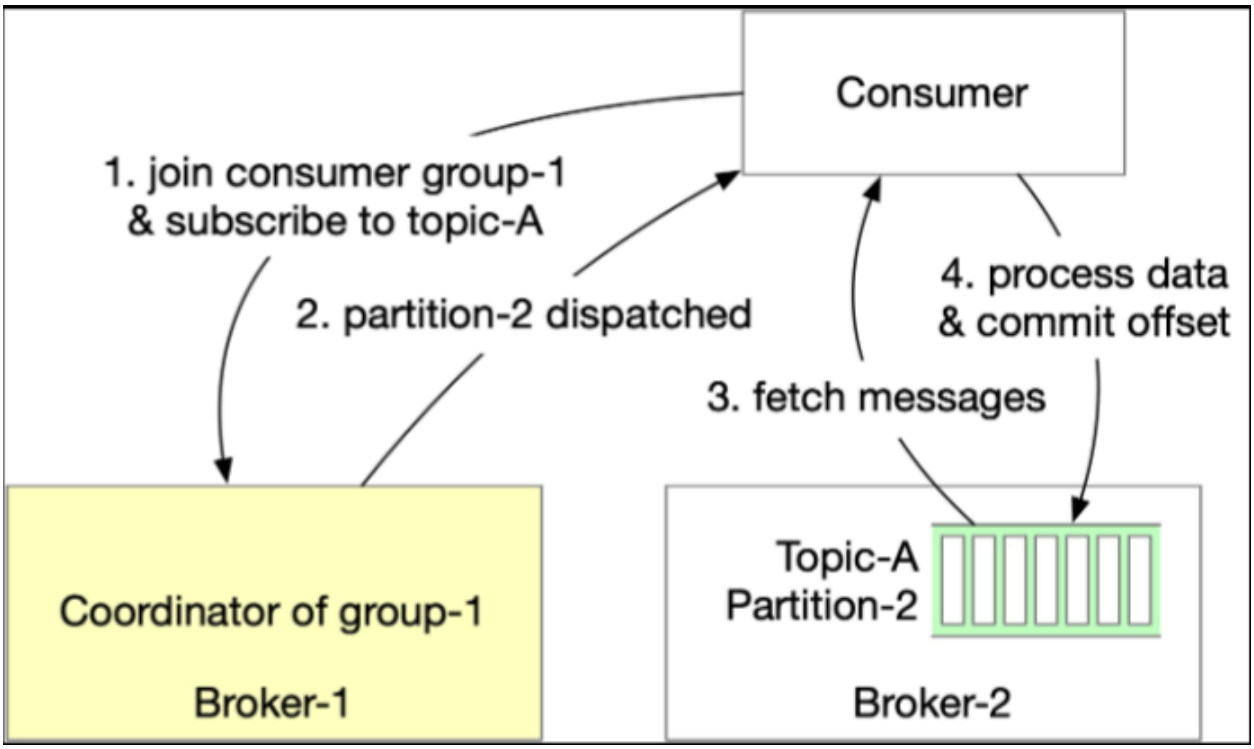
- Why routing layer ? and why routing layer coupled with the producer ?
  - because it helps with availability at the cost of network latency
  - helps distribute load between read and write replicas
  - helps with decoupling the client from the broker using service discovery
  - buffers help with batching

Consumer Flow



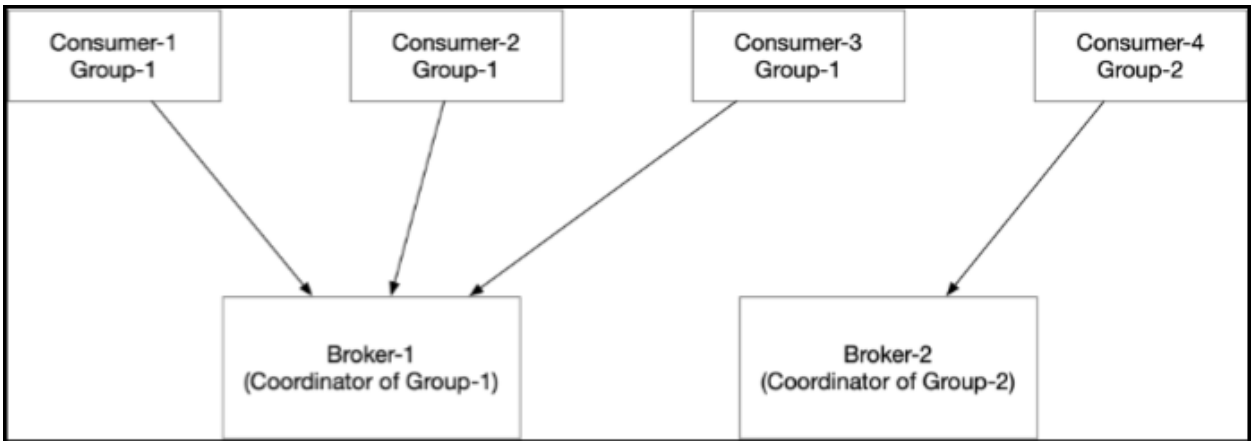
- Consumer specifies its offset in a partition and receives back a chunk of events beginning from that position
- Push vs Pull model

Model	Pros	Cons
PULL model	• Consumers control consumption rate so no overwhelm • more suitable for batch processing because consumer can decide how much to pull at a time • can accommodate both batch & real-time consumers	• consumer might waste resources polling empty queue just waiting for data to arrive
PUSH model	• low latency	• consumer overwhelm possible in-case of impedance mismatch between broker & consumer • difficult to accommodate diverse consumers with varied processing powers

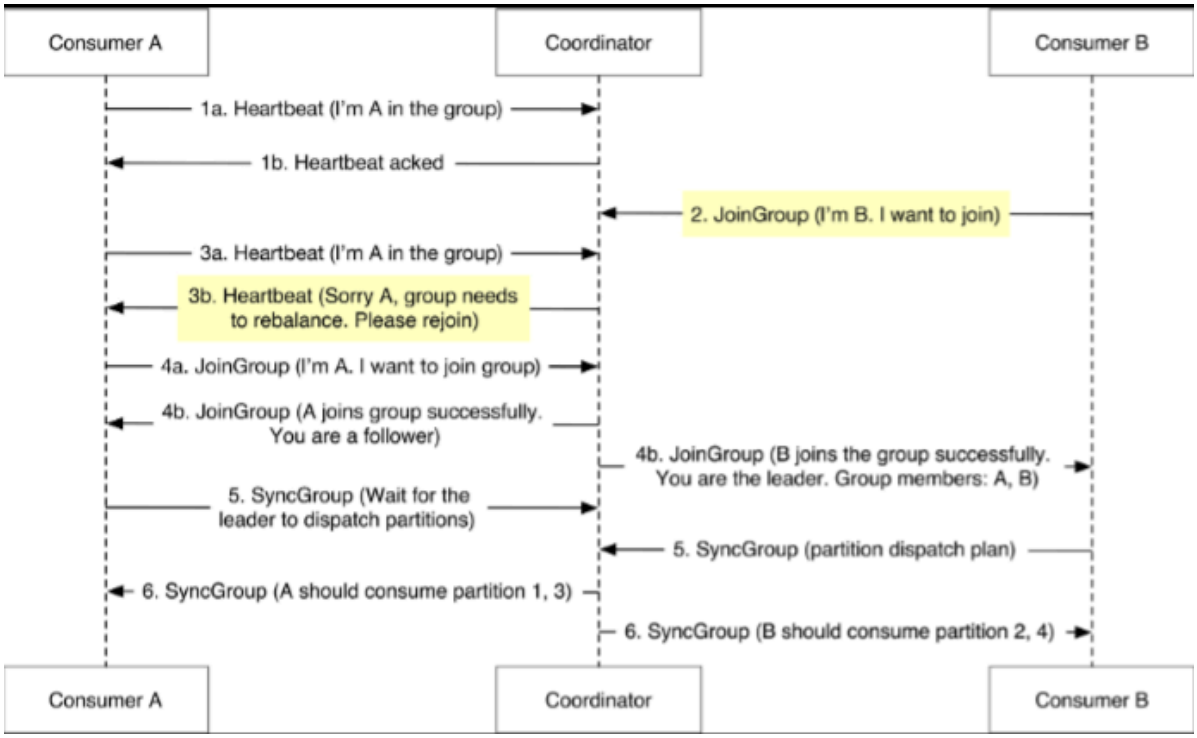


Consumer rebalancing

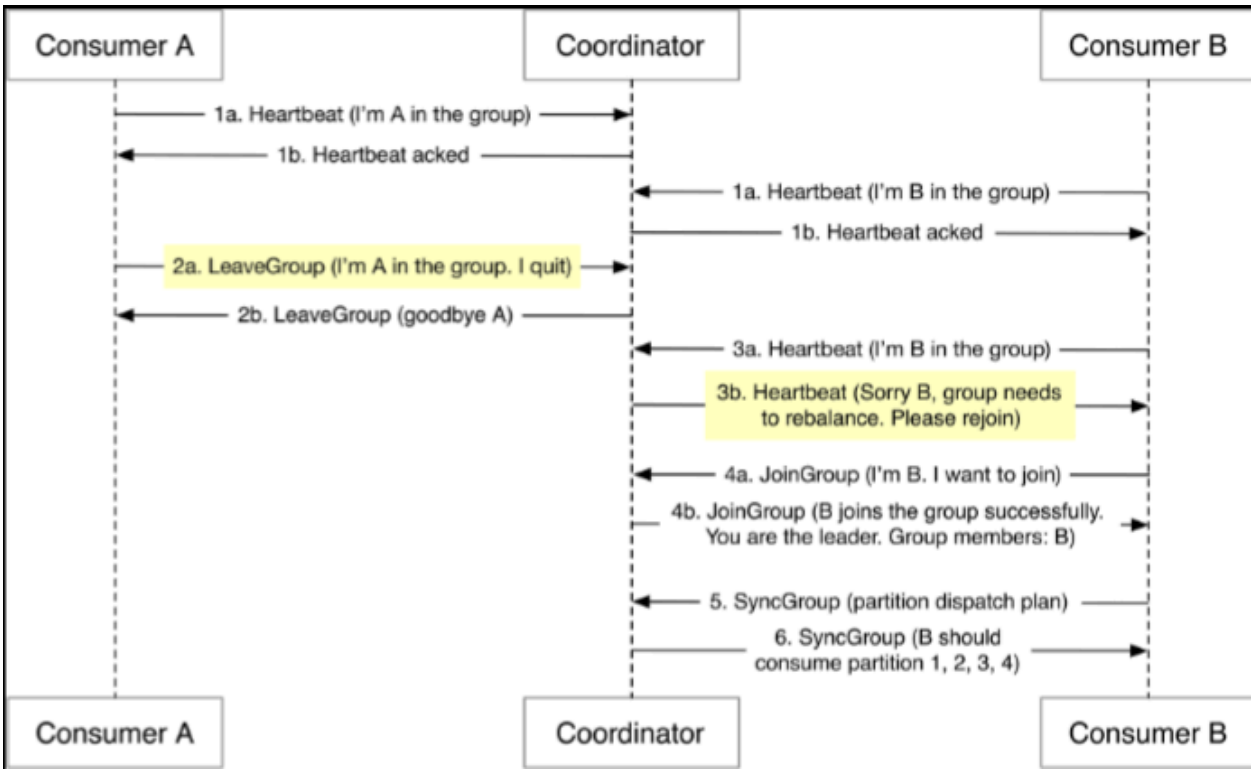
- Decides which consumer is responsible for which subset of partitions
- Role of a coordinator : -
  - usually, one coordinator per consumer group
  - coordinator receives heartbeats from consumers and manages their offset on the partitions
  - each consumer belongs to a group, it finds the dedicated coordinator by hashing the group name
  - coordinator maintains a joined consumer list, when the list changes the coordinator elects a new leader of the group
  - the new leader generates a new partition dispatch plan and reports it back to the coordinator, coordinator will then broadcast the plan to other consumers in the group



- Different scenarios which trigger a rebalancing and how they are managed
  - A new consumer joins the group



  - An existing consumer leaves (similar behavior when an existing consumer crashes, coordinator doesn't receive an ACK and considers the consumer dead)



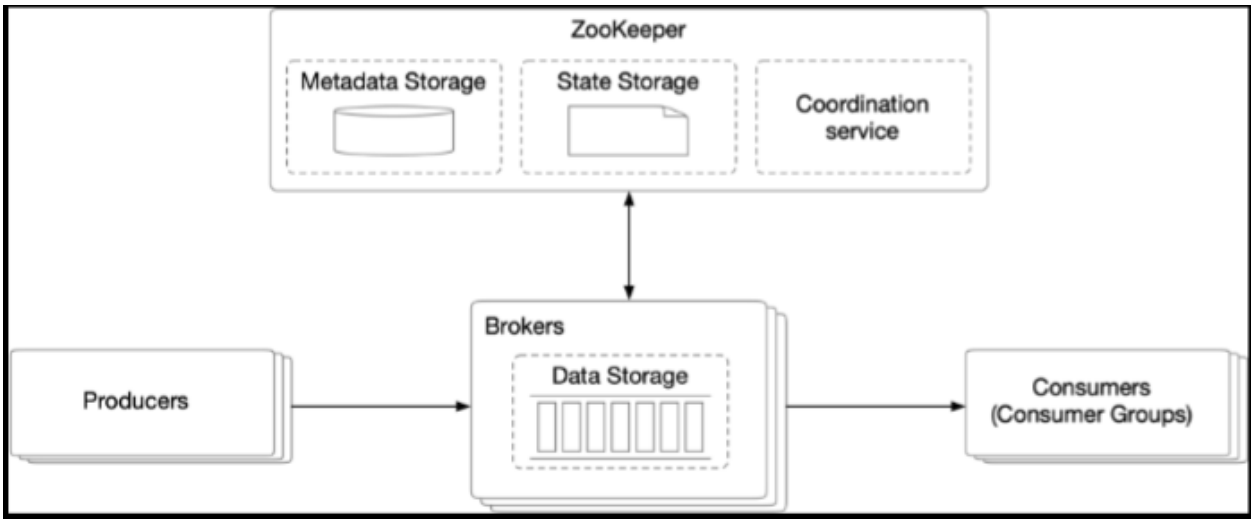
State Storage

- Mapping between partitions and consumers
- Last consumed offset of consumer groups for each partition
- Data access patterns for consumers : -
    - Frequent read & write operations but the volume is not high
    - Data is updated frequently & is rarely deleted
    - Random read & write operations
    - Data consistency is of paramount importance

Metadata Storage

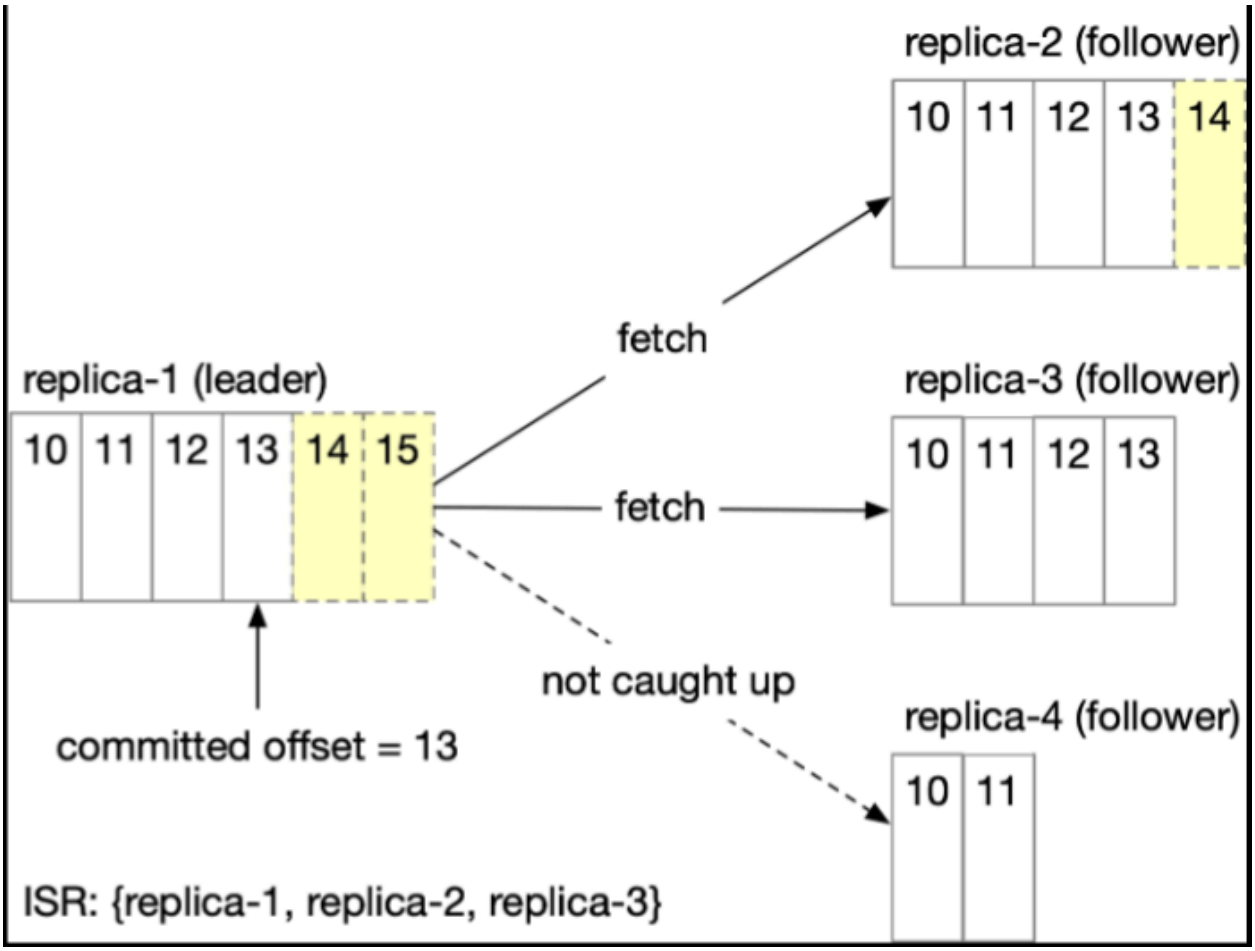
- Stores the configurations & properties of topics, including a number of partitions, retention period, and distribution of replicas ; metadata doesn't change frequently and the volume is small, but it has a high consistency requirement

- Zookeeper can be used for this purpose, it provides a hierarchical key-value store



In-Sync Replicas (ISR)

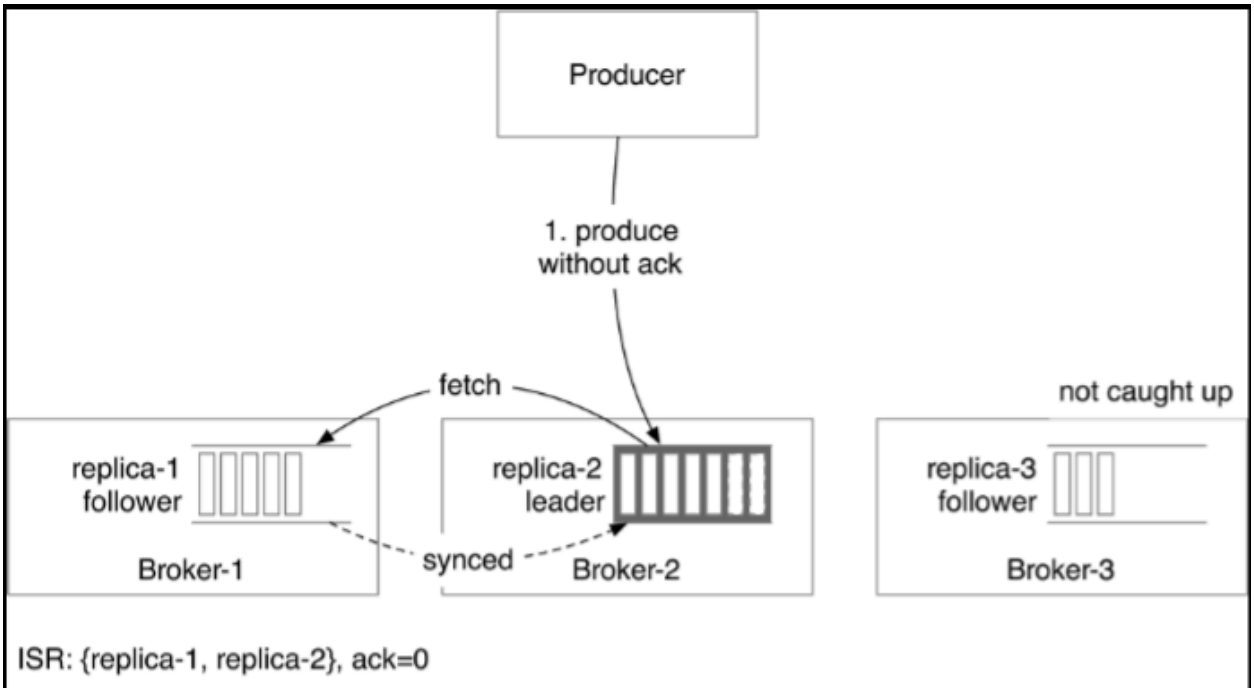
- If `replica.lag.max.messages = 4` then as long as follower is behind the leader by less than 4 messages, it will not be removed from the ISR ; leader is an ISR by default
- In the diagram below, follower replicas 2 & 3 are actually in-sync with the leader and can accept new messages but replica 4 still needs to catch up



Acknowledgement settings

- Producers can choose to receive acknowledgments until the K number of ISRs has received the message, where K is configurable
- ACK = all
  - Gives strongest durability at the cost of taking longer time because we need to wait for the slowest ISR
  - Only effective if the number of ISRs is not below the configured minimum ISR amount configured through `min.insync.replicas`
- ACK = 1
  - Producer receives the ACK as soon as the leader persists the message. Latency is improved because we don't have to wait for data synchronization at the cost of durability in case the leader is lost

- ACK = 0
    - Producer keeps sending messages without waiting for ACKs and it never retries — provides least latency at the cost of potential data loss — useful for logging or metrics collection cases where volume is very high and some data loss is acceptable

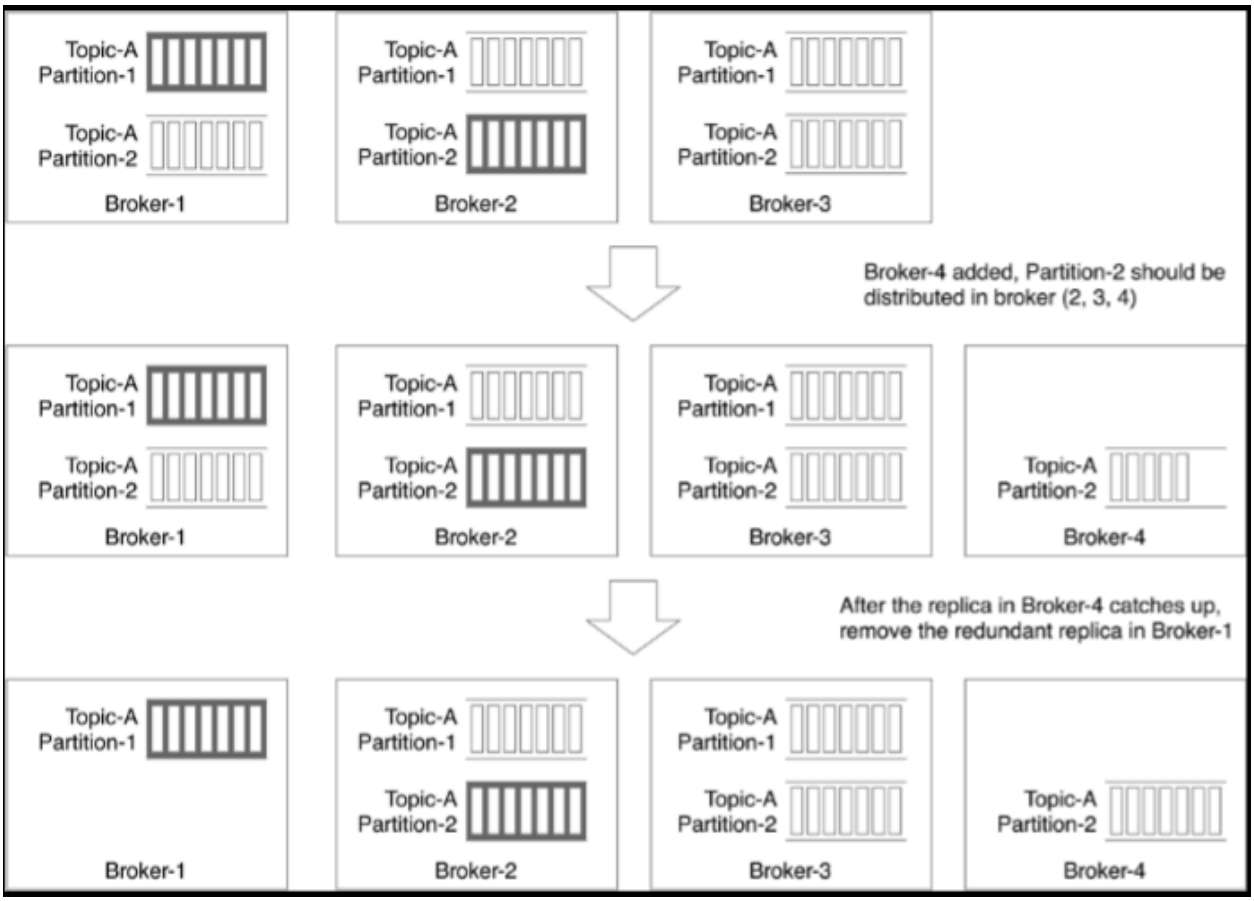


- The consumer can be configured to either read from the leader or closest ISR depending on the business use-case and latency tolerance

Scaling individual components

- Producers
    - Doesn’t need group coordination, just add or remove producer instances
- Consumers
    - Consumer groups are independent of each other, so we can add or remove more consumer groups
    - Inside a consumer group, coordination helps with addition & removal of individual consumers

- Brokers
  - Constraints to respect before we scale : -
    - We must honor minimum number of ISR
    - Data replication within the same node is a waste of resources
    - If all replicas of a partition collapse then the partition data is lost forever, we MAY utilize data mirroring to copy data across data centers



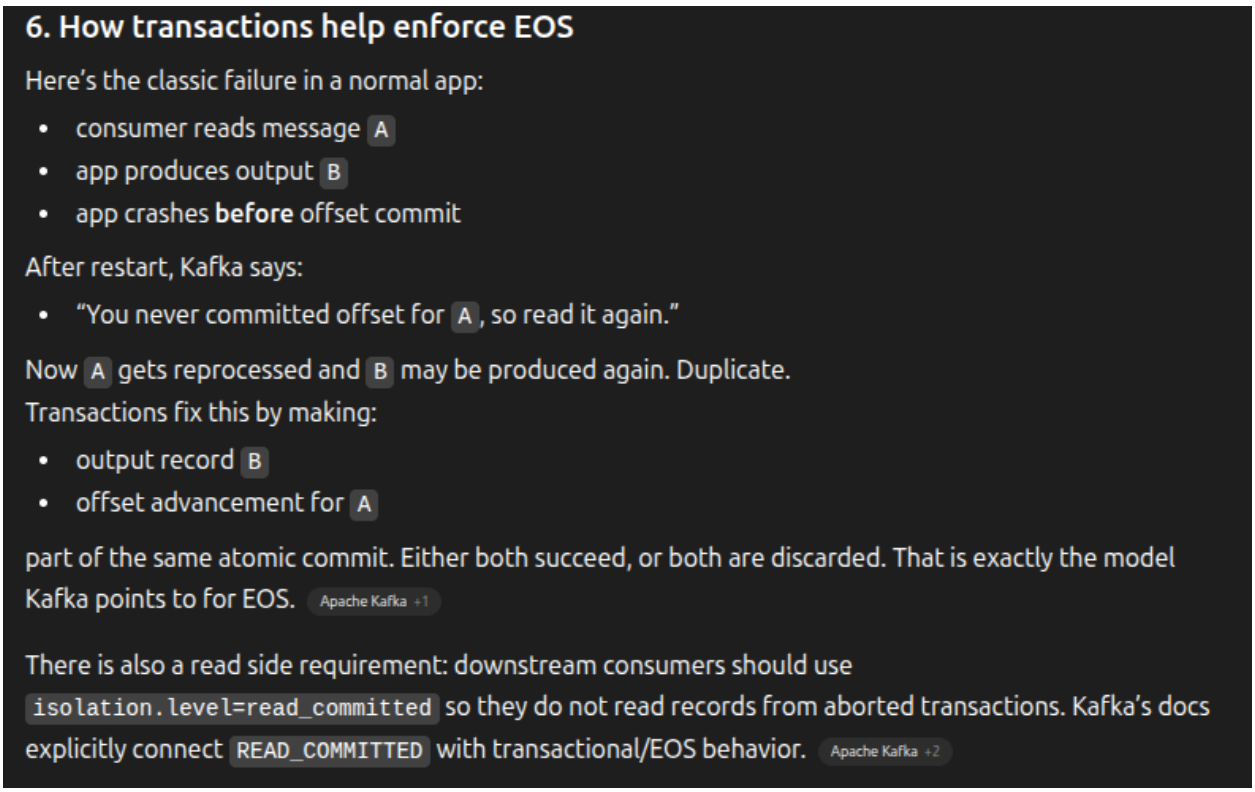
- Partitions
  - Increasing partitions is pretty straight-forward because persistent messages stay in the old partitions so there’s no data migration ; after a new partition is added, new messages will be persisted in all new partitions
  - Decreasing partitions is tricky

- Decommissioned partition can’t be removed immediately because it may still have data which is recently consumed. Only after the retention period passes, data can be truncated and storage space is freed up. Reducing partitions is not a shortcut to reclaiming data space
- During this transitional period, producers only send messages to the remaining partitions, but consumers can still consume from the soon-to-be-dead partition, after the retention period expires consumer groups need rebalancing

Data Delivery Semantics

- At Most Once (no duplicate guarantee)
  - Producer sends a message without waiting for ACKs and doesn’t retry
  - Consumer fetches the message and commits the offset before data is processed, if consumer crashes right after offset commit then the message will not be reconsumed
- At Least Once (no data loss guarantee)
  - Producer will keep retrying unless it receives an ACK (either 1 or all … definitely not 0)

    - Consumer will commit the offset only after the message is successfully processed, if consumer fails to process the message, it will reconsume the message ... if the consumer processes the message but fails to commit the offset, even in that case the message will be reconsumed again
    - With a unique key in each message, a duplicate message can be rejected while writing to a database ... this key uniqueness ensure deduplication while ensuring message delivery
- Exactly Once
    - Mainly used for financial data ; very difficult to implement
    - Can be enforced using Kafka transactions for a Kafka native workflow using the typical consume-process-produce workflow
    - Can be enforced using idempotent producers and consumers when integrating with external downstream components



2. What Kafka transactions are

Kafka **transactions** let a producer group multiple writes into a single atomic unit, and in a consume-process-produce app, they can also include the **consumer offset commit** in that same unit. To use them, the producer is configured with a `transactional.id`, then uses the transactional API: `initTransactions()`, `beginTransaction()`, `commitTransaction()`, or `abortTransaction()`. Kafka's producer docs say `transactional.id` enables transactional delivery across producer sessions, and if it is set, idempotence is implied. Apache Kafka +2

So the producer is basically saying:

> "These output records and this offset commit either all become visible together, or none of them do."

That is the key jump from plain flow to transactional flow. Apache Kafka +1

- Kafka streams is just a library which makes development of streaming services a lot easier, it doesn't bring any added feature to core kafka feature-set
    - Streams make EOS much simpler

Kafka Streams flow

With Streams, you usually just declare the topology and set:

properties

```
processing.guarantee=exactly_once_v2
```

Then Streams manages the internal transaction/commit behavior for you. Kafka's docs say that for exactly-once processing, committing progress means committing a transaction that both saves the position and makes output visible to `read_committed` consumers. Apache Kafka +2

7. Tiny mental model

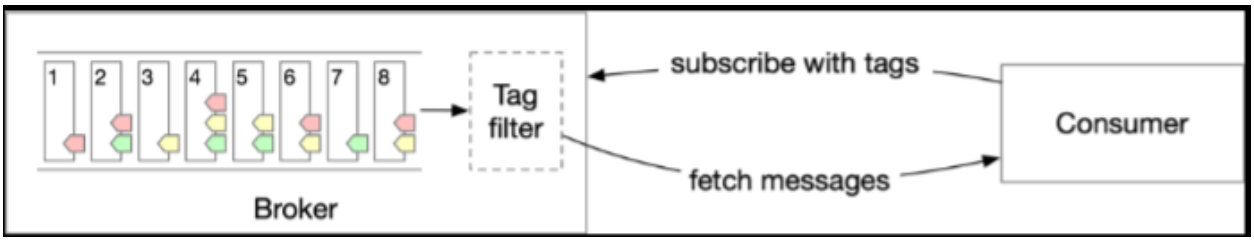
Use this:

- **normal producer/consumer**
  "I send and consume records, but crash boundaries can create duplicates."
- **idempotent producer**
  "Retries won't duplicate the same write."
- **transactional producer**
  "A group of writes, plus offset progress, can commit atomically."
- **Kafka Streams**
  "A framework that does the poll/process/state/output/commit dance for me, and can enable EOS with `exactly_once_v2`."

That is the clean hierarchy. Apache Kafka +2

Message filtering

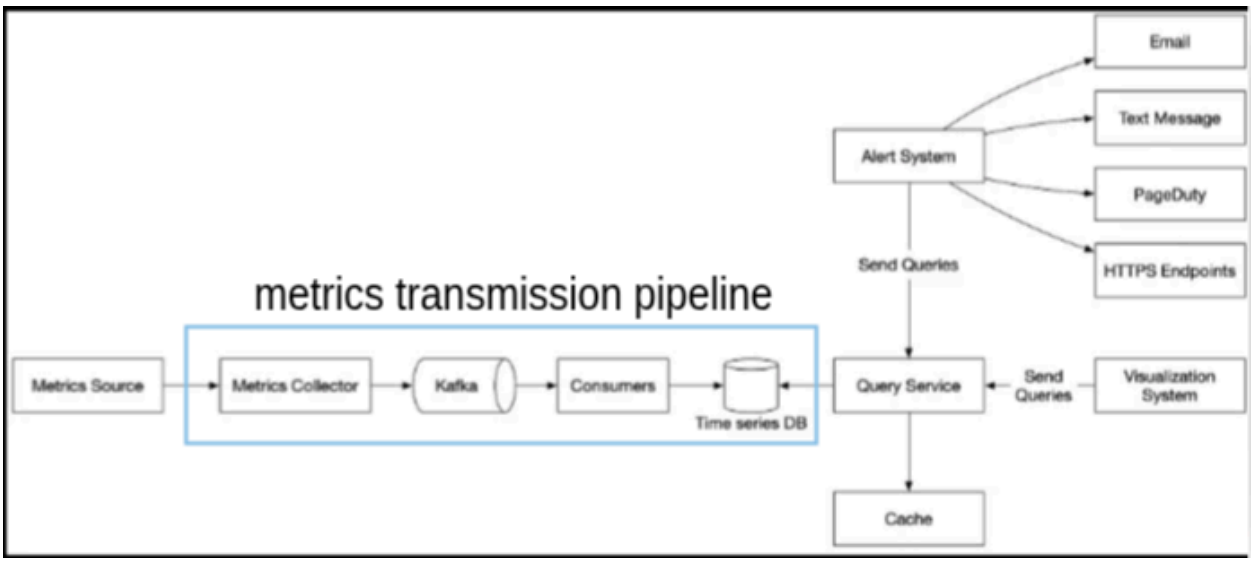
- Filtering is better handled on broker side, this is because allowing consumers to read unnecessary data is an unnecessary increase in traffic
- The filtering logic must not extract the payload because this process will slow down the broker
- Best approach is to include a tag in the metadata of the message which can be used to filter messages in that dimension
- Multiple tags can support multi-dimensional filtering



Metrics Monitoring & Alerting system

- Functional requirements
  - 100 million DAU

    - 1000 server pools, 100 machines per pool & 100 metrics per machine
    - 1 year data retention, raw for 7 days — 1 minute resolution for 30 days — 1 hour resolution for 1 year
- Non functional requirements
    - Scalability — able to accommodate growing metrics & logs
    - Low latency — for queries to generate dashboards & alerts
    - Reliability — can't miss critical alerts
    - Flexibility — integrate new ever-changing technologies in future



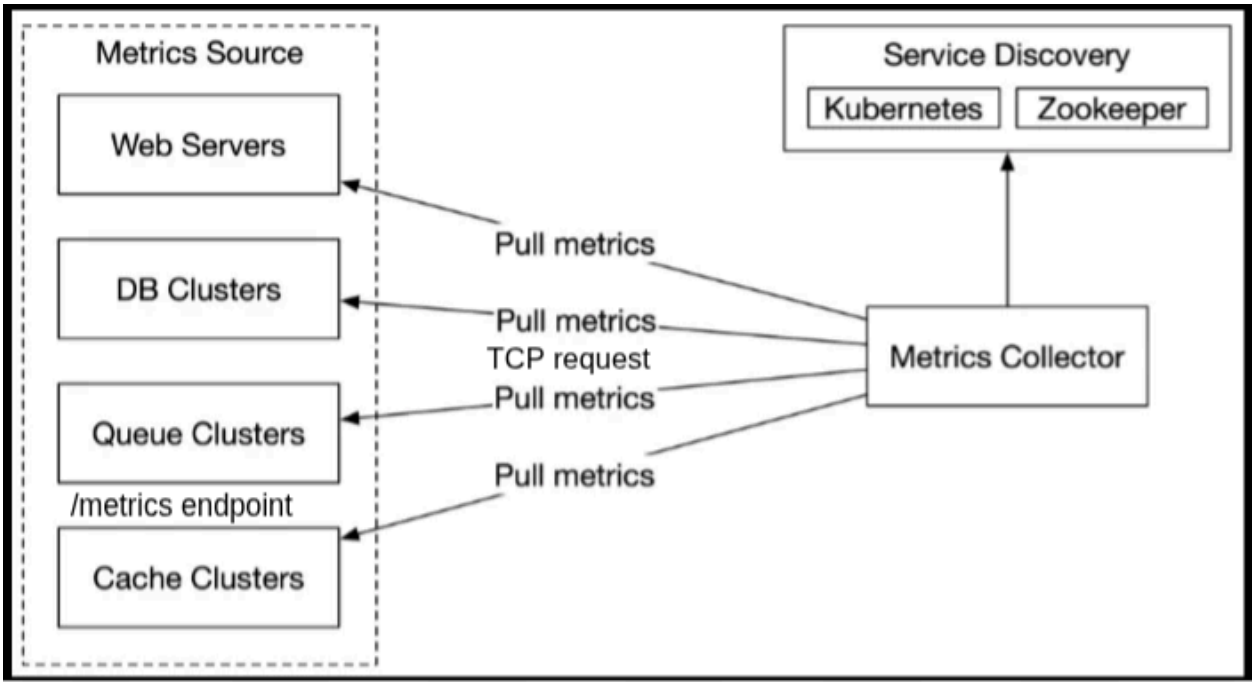
Data Model

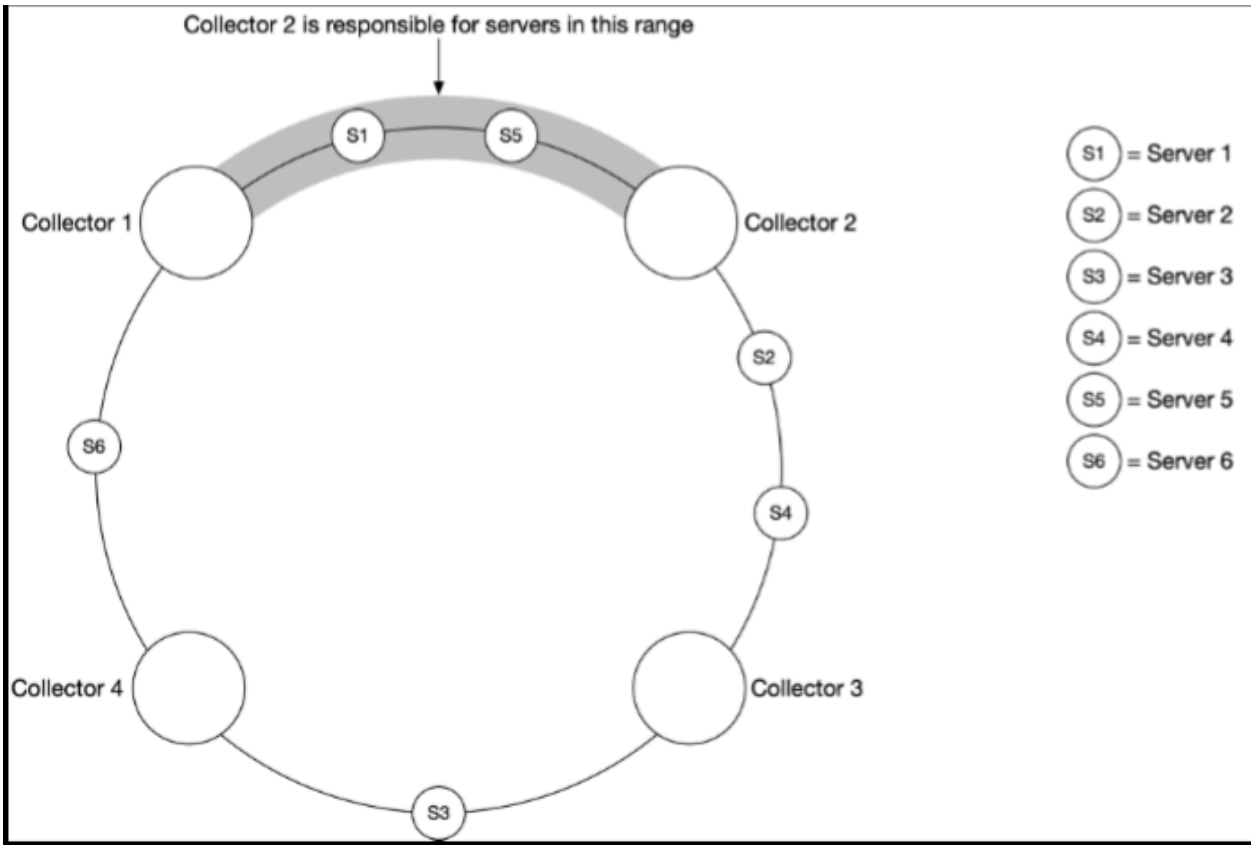
Name	Type
A metric name	String
A set of tags/labels	List of <key:value> pairs
An array of values and their timestamps	An array of <value, timestamp> pairs

- Multiple labels can have the same metric and multiple metrics can have the same label ... this is to simplify aggregate queries
- System is constantly write heavy with read spikes
- We cannot use general purpose relational & NoSQL DBs for timeseries data, they would provide horrible performance while querying — specific DBs like Prometheus & InfluxDB are used to store timeseries data

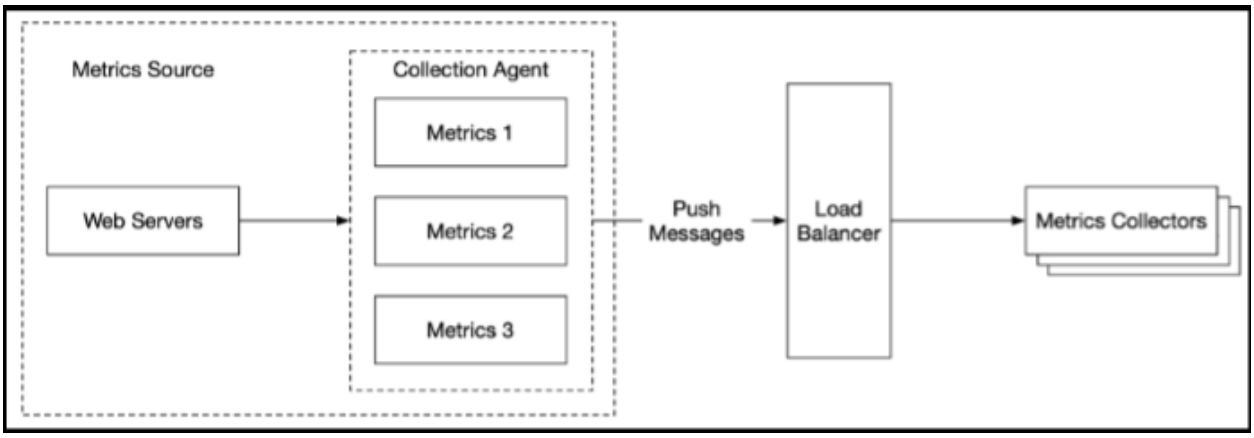
Metrics Collection

- Pull model used by Prometheus
  - At our scale, one metric collector can't poll all the machines, so each metrics collector communicates with a service discovery system to identify machines on its hash-ring and pulls the machine's metadata
  - Consistent hashing can be used to ensure only one collection server is pulling metrics from one machine — to eliminate duplicate metrics and remove pollution





- Push model used by CloudWatch
  - Requires us to configure an agent on the machines which can aggregate the metrics per machine and then stream the metrics to the collection server
  - Collection server must be behind a loadbalancer to ensure metrics are not lost during peak traffic hours
  - Most agents have a buffer which can enable batching, aggregation & back-log / retry queue for metrics if the collection server is busy



Metrics transmission pipeline

- Scaling the collection server is very straightforward, simply put it behind an autoscaling group
- Scaling the DB is tricky — if the DB uses event-trigger based scaling then we might run into a time-window where the DB is full and potentially loses data
- Best option to scale the transmission pipeline is to decouple the collector service with the DB either using a system like Kafka or buffer queues or Flink to ensure data can be held in queues while the DB scales instead of suffering data loss

Aggregation

- Collection agent — installed on the client machine & supports simple aggregation logic only
- Ingestion pipeline — use systems like Flink to reduce volume of data being written to the DB
- Query side — can cover the entire dataset at the cost of high query time

Query Service

- This isn’t really necessary because most visual & alerting systems have inbuilt plugins to run queries

- BUT, if you must, you can create a cluster of servers that have access to the entire dataset on the timeseries DB
- You can also couple query servers with a cache layer to reduce query load and store frequent query results

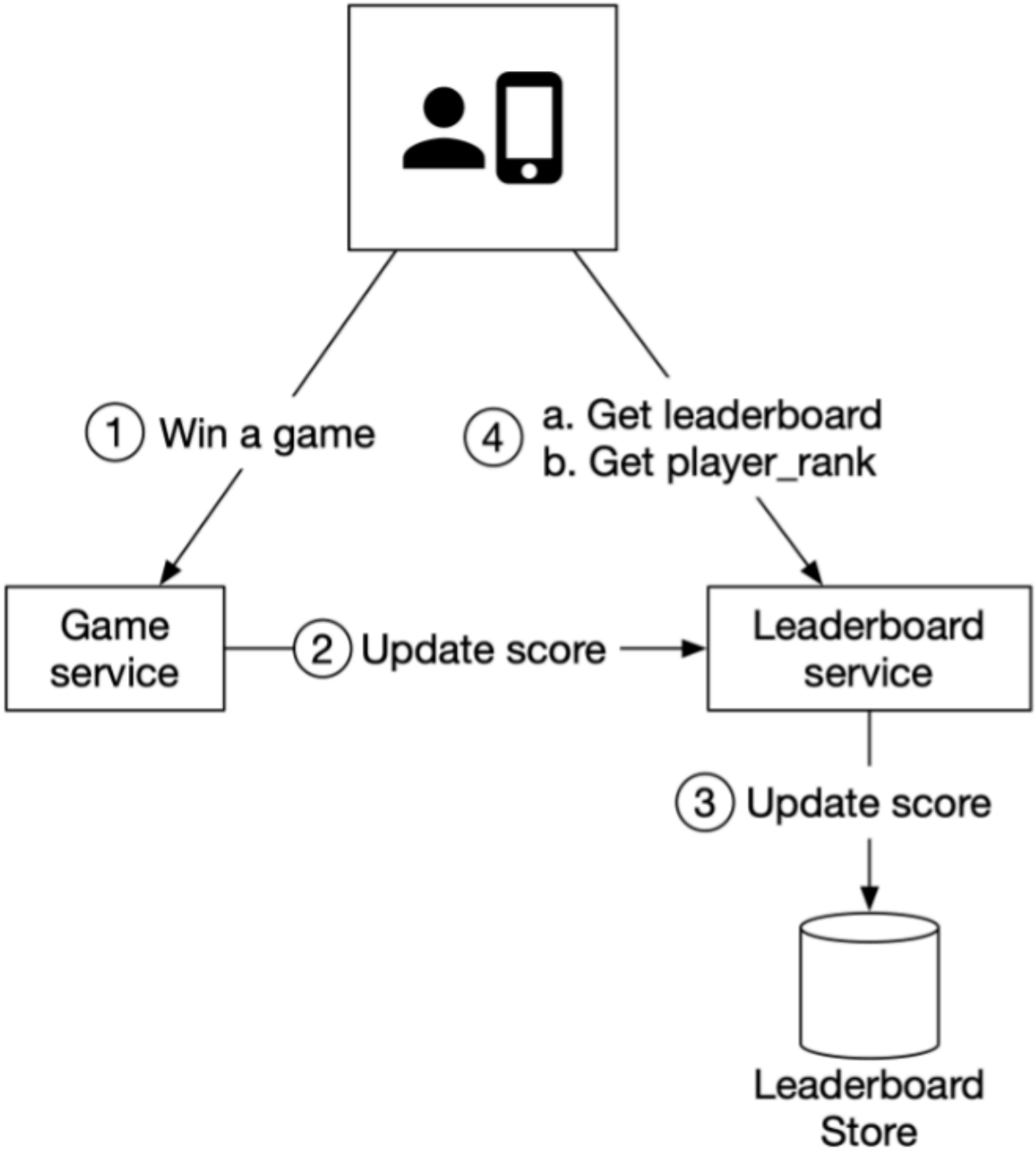
Space Optimizations

- Data encoding & compression — yeah, simply compress and encode the data in the timeseries DB to improve performance
- Downsampling — convert high resolution data to low resolution data; this reduces load on the server

Alert System & Visual systems

- Honestly, just buy them off the shelf
- BUT, if you must, you can create an alert manager service which sits in either of these 2 places
    - Between the transmission pipeline & timeseries DB — it can be added as a consumer to the kafka queues & trigger alerts using metrics that meet a certain threshold, because we don't need to store data on this service, we can simply truncate or remove the data after a set time period (this is push configuration)
    - Between DB & query service or maybe even on the query service — server can open a long poll connection with the DB and keep running a query at a set cadence ... the moment a threshold is breached we can send out the alert (this is pull configuration)
    - The alert store must have a key-value store to keep the state of an alert and ensure that at-least one notification is sent out per alert

Gaming Leaderboard

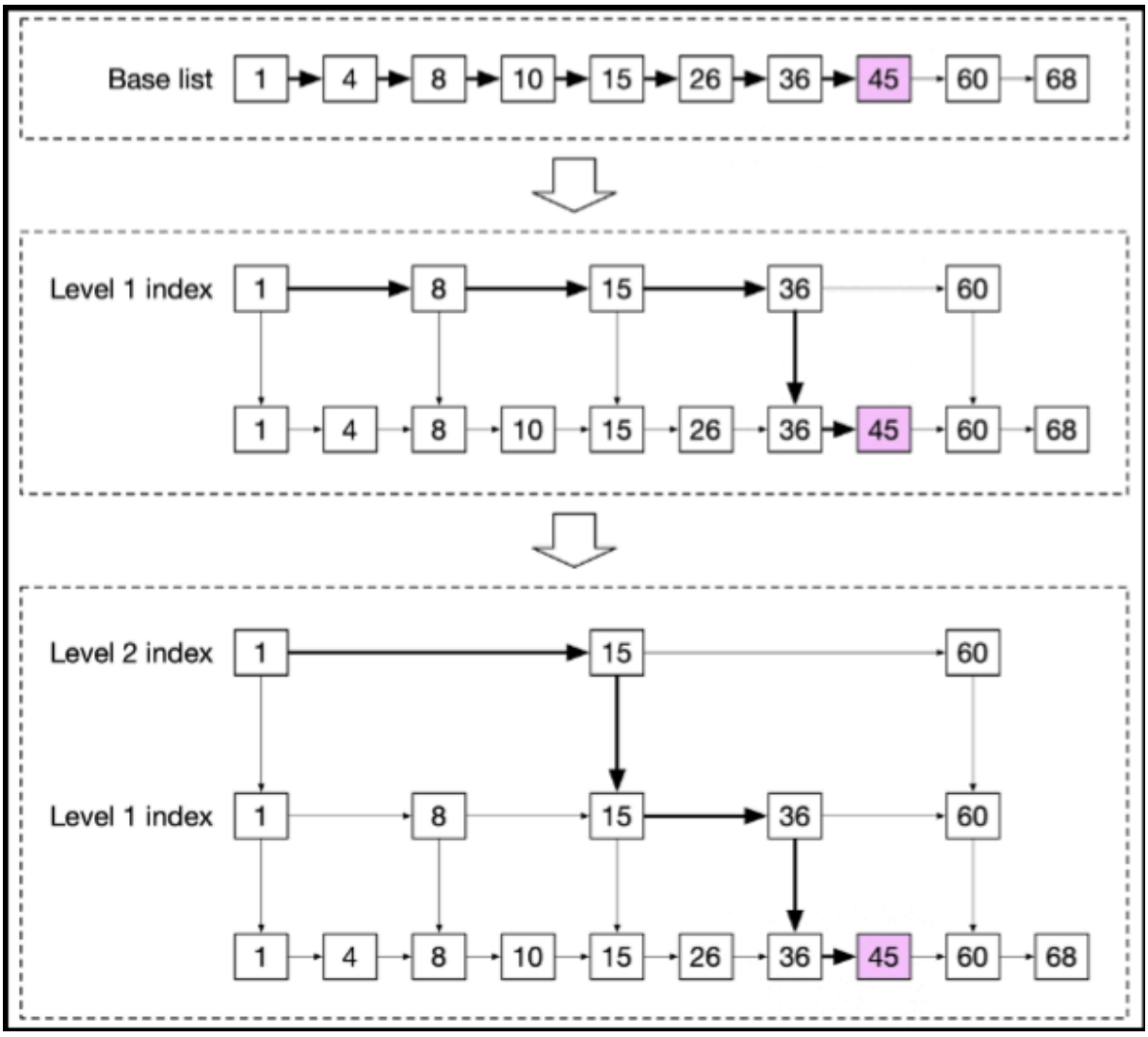


- Core component can be designed mainly using 2 options : -
  - Redis sorted set : each member of a sorted set is associated with a score, members of set must be unique but scores may repeat, the score is used to rank the sorted set in ascending order

- NoSQL approach : DynamoDB could be a good fit because it is optimized for writes and efficiently sorts items within the same partition by score

Sorted set deep dive

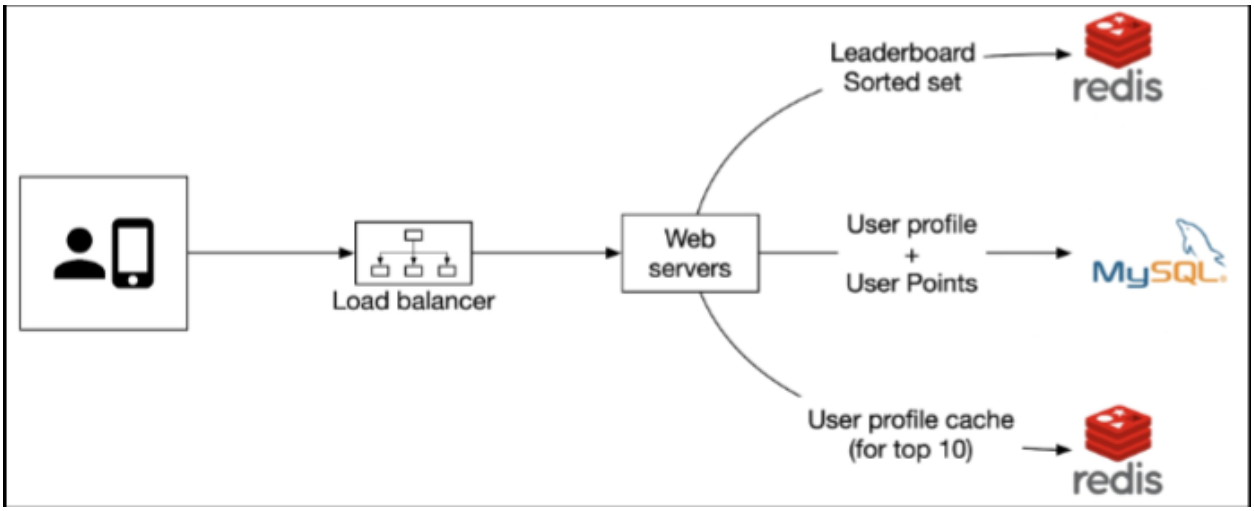
- High level idea revolves around a `skip list` which allows for fast search, it consists of a base sorted linked list and multi-level indices
- Think binary search but at each level, the parent skips its child instead of skipping the entire dataset — we add a level 1 index that skips every other node, and then a level 2 index that skips every other node of the level 1 indexes. Repeat this logic until we have level x indexes that jump approximately to the middle



- Redis implementation revolves around 3 main functions
  - ZADD : insert the user into set if they don't exist yet, else update the score
  - ZINCRBY : increment the score of a user, if user doesn't exist assume base score to be 0 and insert with updated value
  - ZRANGE / ZREVRANGE : fetch a range of users sorted by score
  - ZRANK / ZREVRANK : fetch the position of any user sorted in ascending/descending order
- You can also fetch a defined number of users ranked above & below our target user using the ZREVRANGE query — if user Mallow007's rank is 361 and we want to fetch 4 players above and below them, we can use this query : -

```
ZREVRANGE leaderboard_feb_2021 357 365
```

- You must configure your redis instance with read replicas because if such a large node goes down, bringing it back up will take a lot of time



- Best mechanism would be to use cloud serverless functions behind an API gateway — we just offloaded all our scaling considerations onto the cloud and don't have to worry about them

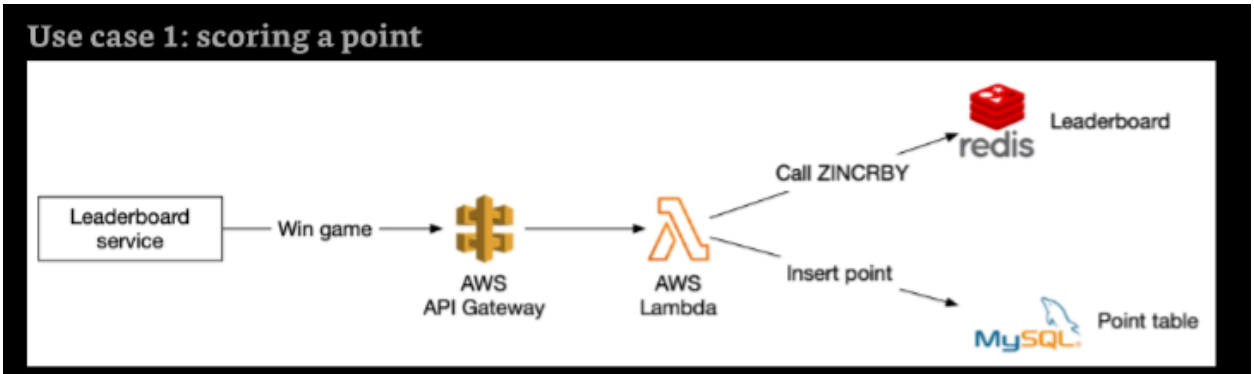


Figure 16 Score a point

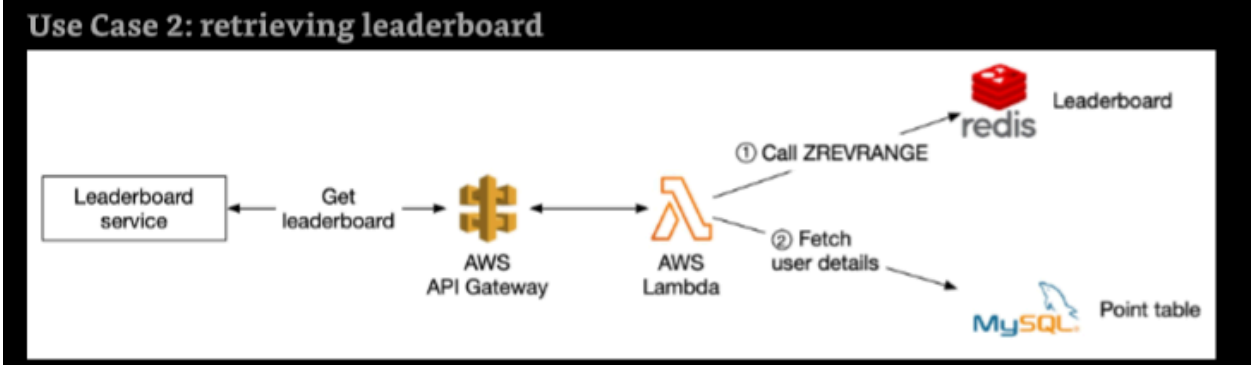
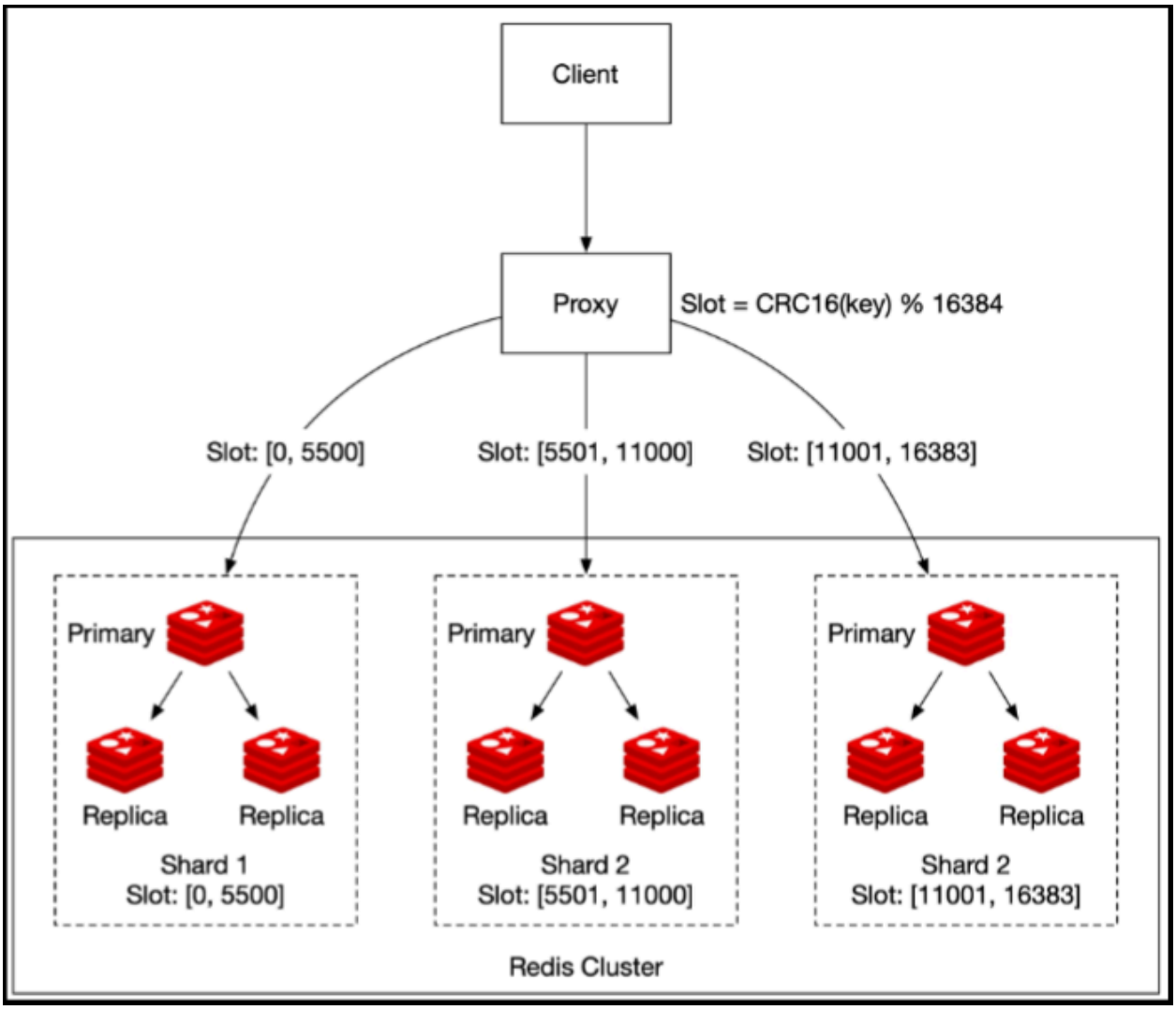


Figure 17 Retrieve leaderboard

- Scaling Redis — requires data sharding
  - Fixed partitions
    - When we are inserting or updating the score for a user, we need to know which shard they are in. We could do this by calculating the user's current score from the MySQL database. This can work, but
    - Another more performant option is to create a secondary cache to store the mapping from user ID to score.
    - We need to be careful when a user increases their score and moves between shards. In this case, we need to remove the user from their current shard and update the secondary cache.

Figure 18 Fixed partition

- Hash partitions
  - Redis cluster provides a way to shard data automatically across multiple Redis nodes. It doesn't use consistent hashing but a different form of sharding, where every key is part of a hash slot
  - Retrieving the top 10 players on the leaderboard is more complicated. We need to gather the top 10 players from each shard and have the application sort the data
  - When we need to return top K (very large number) results on the leaderboard, the latency is very high because a lot of entries are returned from each shard and need to be sorted
  - Latency is very high because we have to wait for the slowest partition, in case we have too many partitions



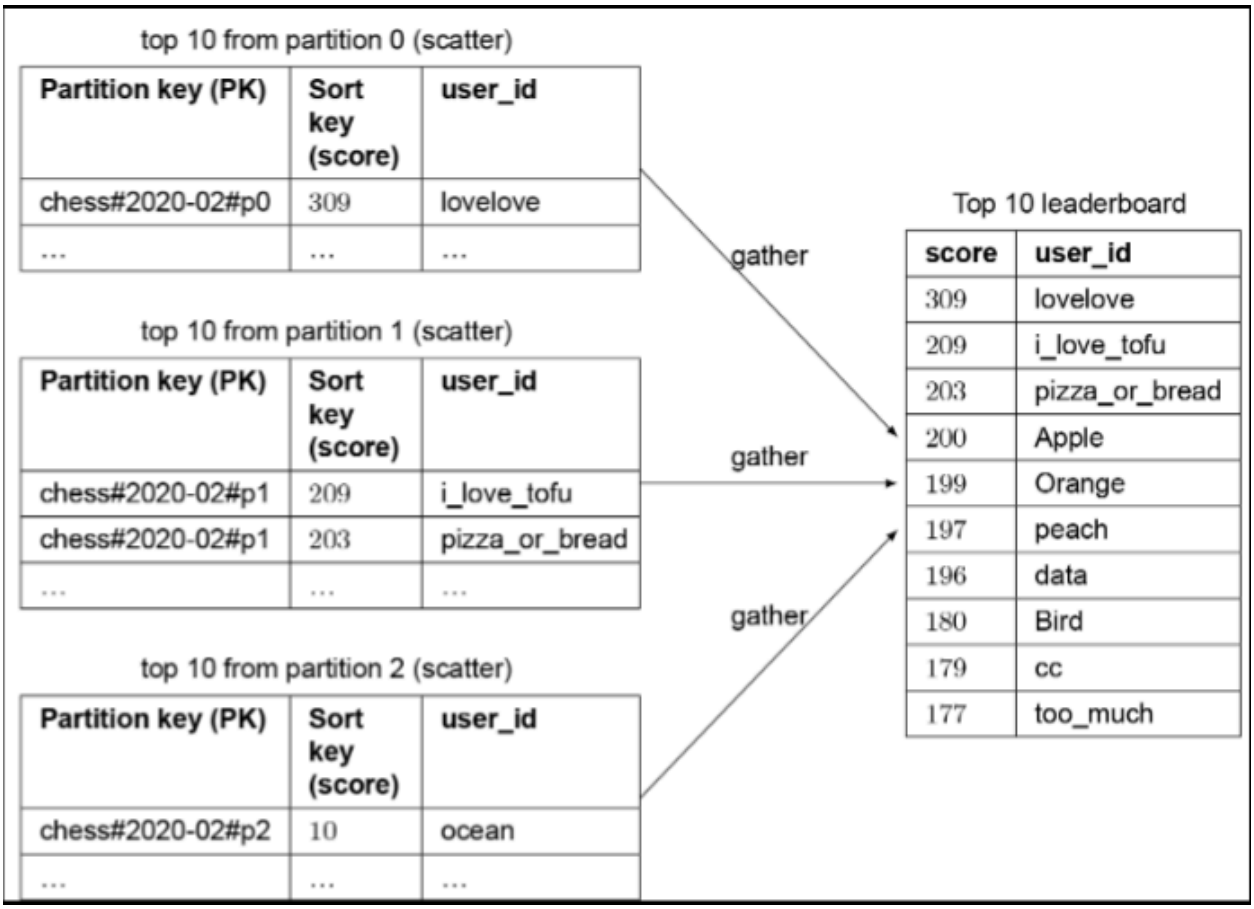
NoSQL deep dive

- For efficient access to data, we need to leverage global secondary indices
- GSI contains a selection of attributes from the parent table, but they are organized using a different primary key — we need to couple this with write sharding

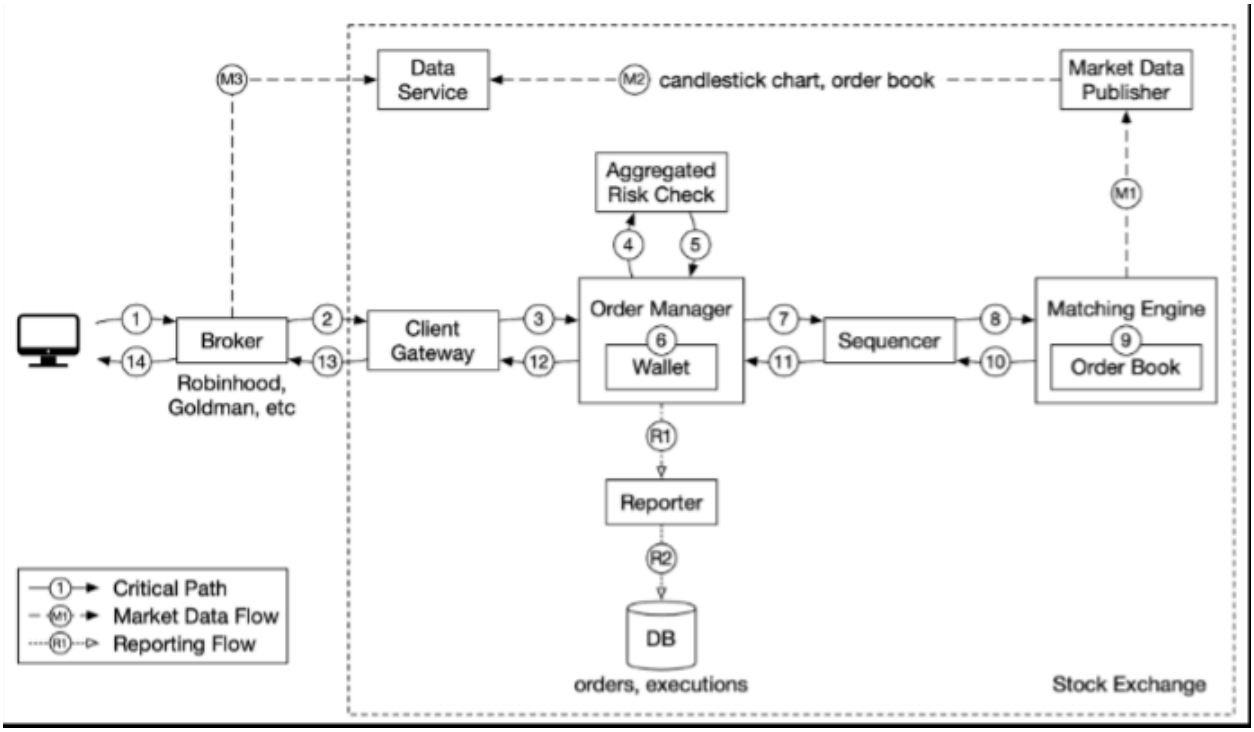
- WTF is write sharding ?
  - DynamoDB splits data across multiple nodes using consistent hashing based on partition key, we need to structure the data so that data is evenly distributed across partitions — this is to prevent hot partition problems
  - We can split data into N partitions and append a partition number (user_id % number_of_partitions) to the partition key

Global Secondary Index		Attributes		
Partition key (PK)	**Sort key (score)**	**user_id**	**email**	**profile_pic**
chess#2020-02#p0	309	lovelove	love@test.com	https://cdn.example/3.png
chess#2020-02#p1	209	i_love_tofu	test@test.com	https://cdn.example/p.png
chess#2020-03#p2	103	golden_gate	gold@test.com	https://cdn.example/2.png
chess#2020-02#p1	203	pizza_or_bread	piz@test.com	https://cdn.example/31.png
chess#2020-02#p2	10	ocean	oce@test.com	https://cdn.example/32.png
...	...	...	...	...

  - Remember Beirut ? write sharding comes at the cost of read complexity because the reader would have to run complex merge queries across partitions to return aggregate results
  - This merge dance performed by the reader is called a “scatter-gather” approach — If we assume we had 3 partitions, then in order to fetch the top 10 leaderboard, we would fetch the top 10 results in each of the partitions (this is the “scatter” portion), and then we would allow the application to sort the results among all the partitions (this is the “gather” portion)

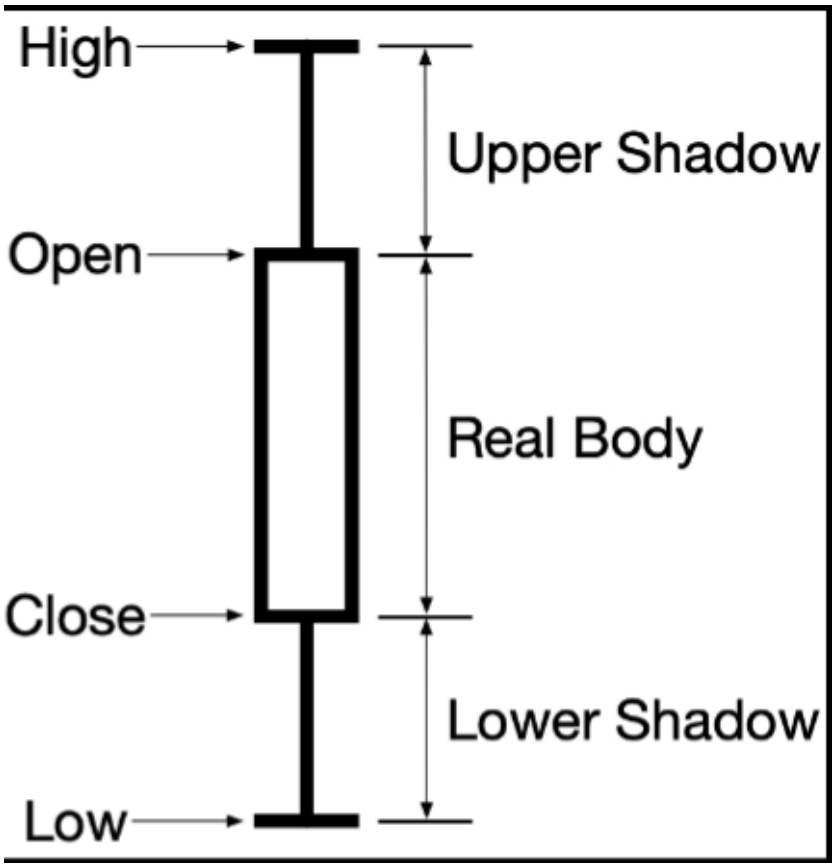


Stock Exchange



Lingo

- Broker — for our purposes, they simply provide an interface to interact with the exchange
- Institutional Client — large volumes using specialized software, each institution has its own requirements, pension funds trade less frequently but with huge volumes and they need order splitting software … market makers trade in huge frequency and require low latency to earn using commission rebates
- Limit Order — buy or sell order with a fixed price, partial or delayed match possible
- Market Order — executed on market price, sacrifices cost to guarantee execution
- Candlestick chart — shows market's open, close, high & low price for a time interval



- FIX protocol — financial information exchange protocol, vendor neutral communications protocol to exchange securities transaction information

```
8=FIX.4.2 | 9=176 | 35=8 | 49=PHLX | 56=PERS | 52=20071123-05:30:00.000 |
11=ATOMNOCCC9990900|20=3|150=E|39=E|55=MSFT|167=CS|54=1|38=15
|40=2|44=15|58=PHLX EQUITY TESTING|59=0|47=C|32=0|31=0|151=15|
14=0|6=0|10=128|
```

Market Data Levels — 3 levels

- L1 includes best bid price, ask price & quantities

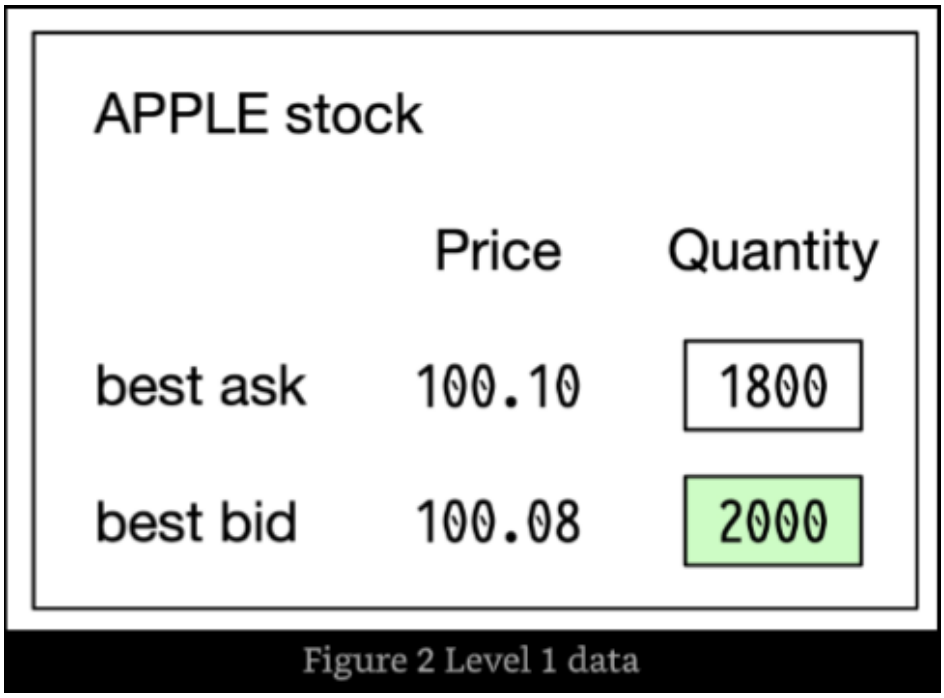
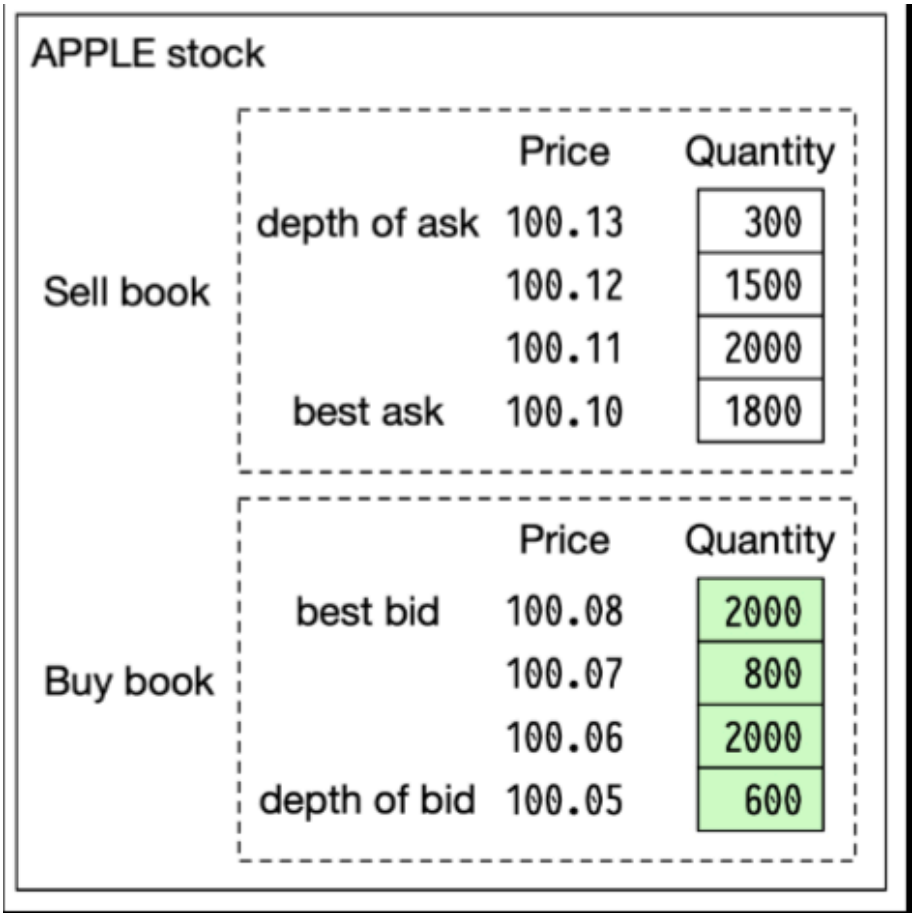


Figure 2 Level 1 data

- L2 includes more price levels than L1



- L3 includes price levels & queued quantity at each price level

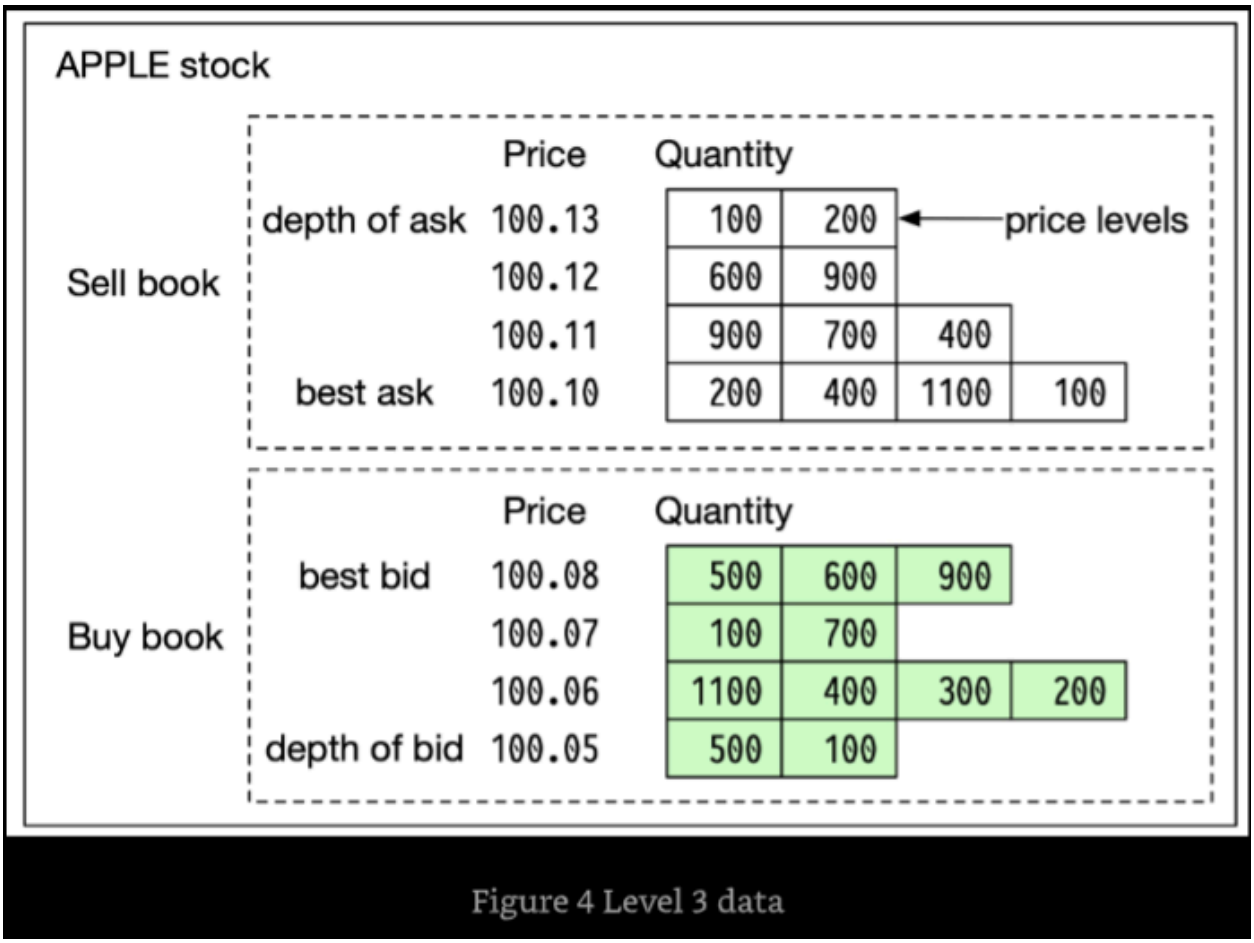


Figure 4 Level 3 data

Execution flow

There are mainly 3 flows concerning a trade

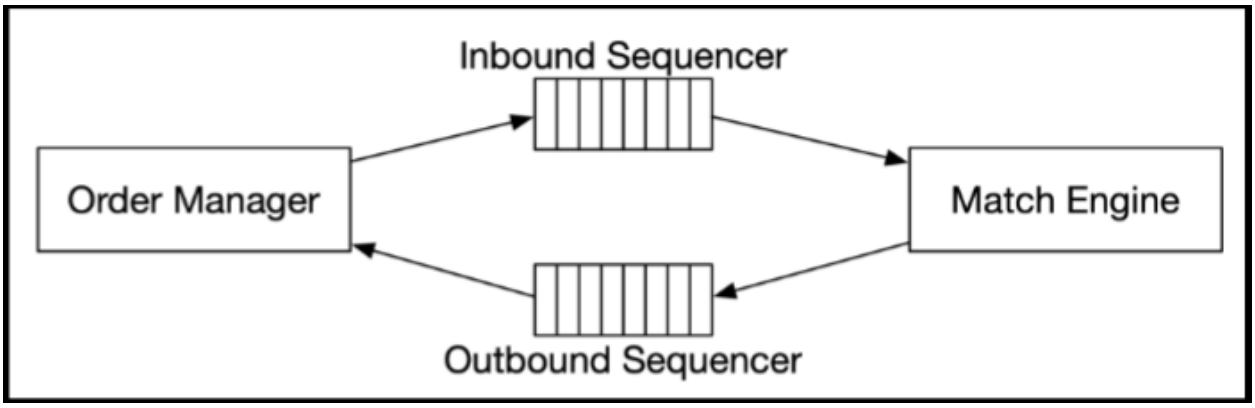
- Trading flow
- Market Data flow
- Reporting flow

Trading Flow

- Matching engine
  - Maintain the order book for each ticker
  - Match but & sell orders
  - Distribute the execution stream as market data

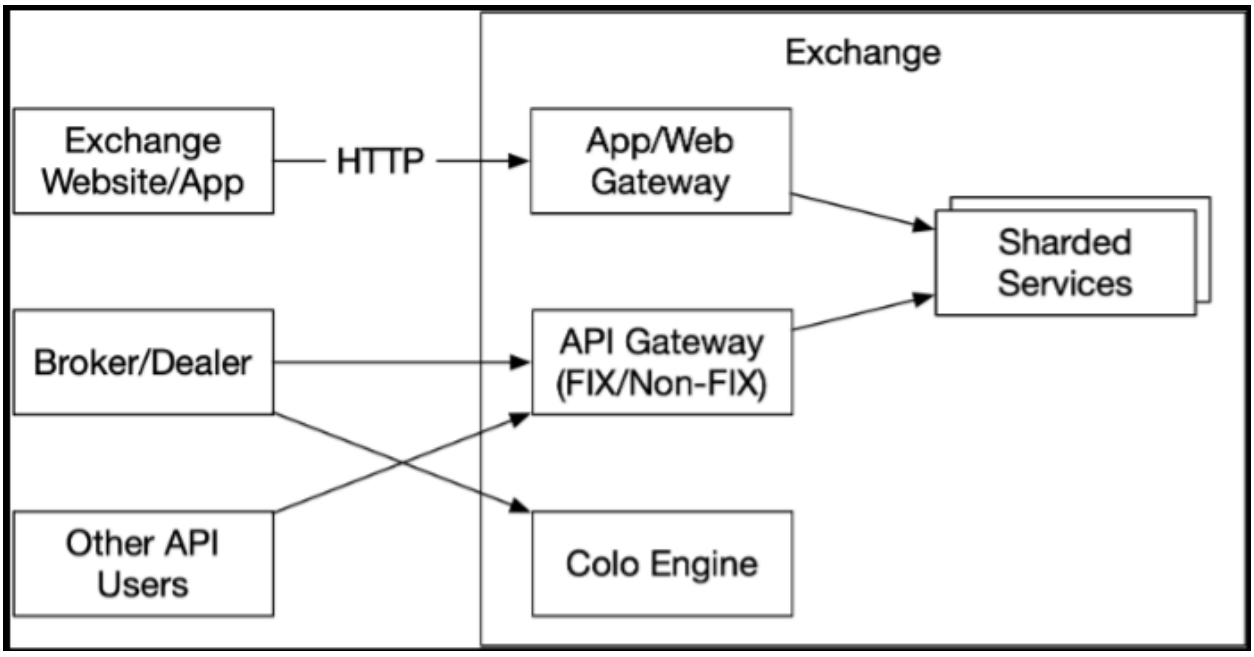
- Must be highly available & able to produce matches in a deterministic order

- Sequencer
  - Key component which makes the matching engine deterministic
  - Sequencer has an outbound and an inbound instance used to stamp every incoming order with a sequence ID & also stamps every pair of execution completed by the machine
  - Must provide : -
    - Timeliness & fairness
    - Fast recovery & replay
    - Exactly once guarantee
  - It also functions as a message queue for incoming orders to the matching engine & executions going back to the order manager



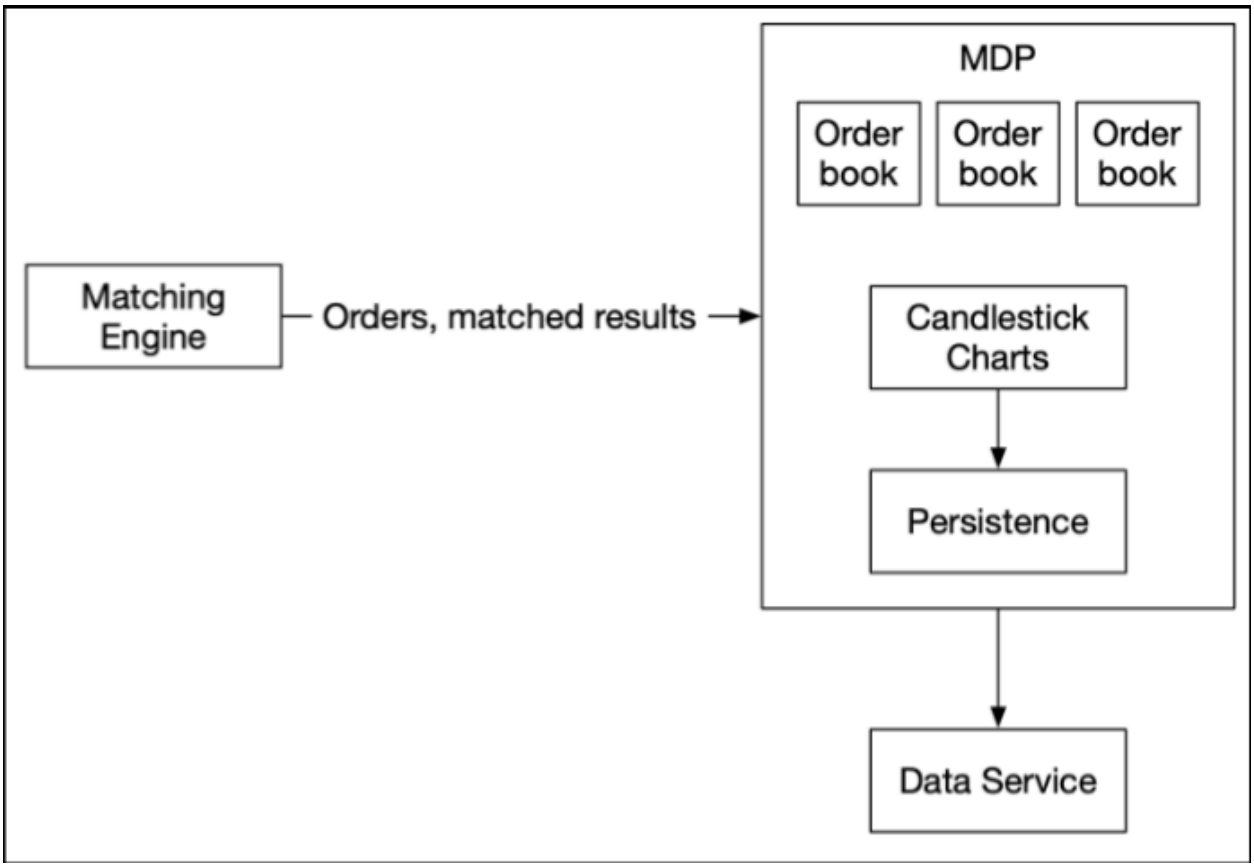
- Order Manager
  - Receives inbound orders from the client gateway & performs risk checks
  - Checks the order against the user's wallet & verifies that there are sufficient funds to cover the trade
  - Order manager might only send the required attributes to the sequencer to reduce traffic on the sequencer

    - The order manager also receives the executions for filled orders and returns to broker via the client gateway
    - Maintains the current state of orders and state transition could be the most complex challenge in this system
- Client Gateway
    - Gateway is on the critical path & is latency sensitive, it should stay lightweight
    - Different type of client gateways for retail & institutional clients
    - It passes orders to the correct destination destinations as quickly as possible



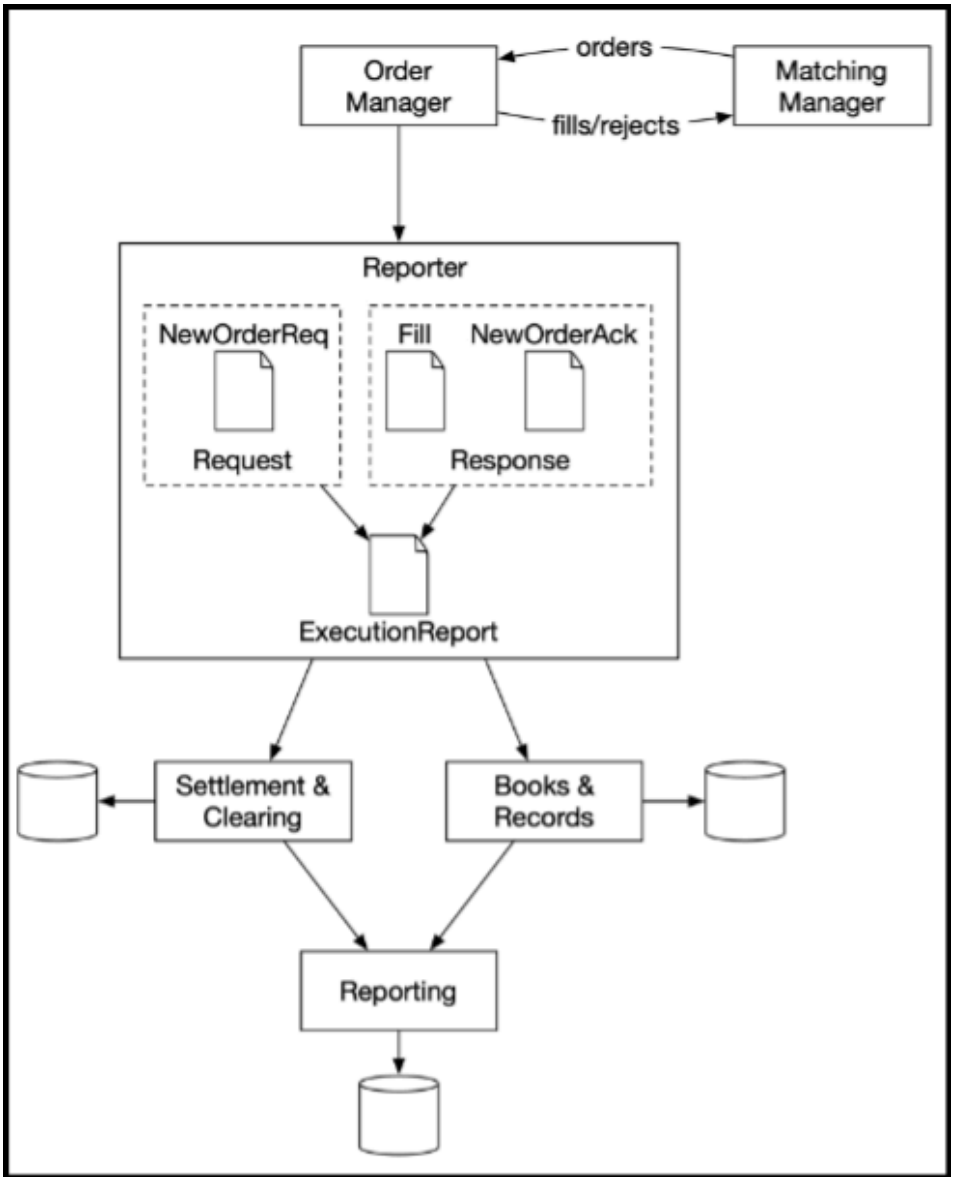
Market data flow

- The market data publisher (MDP) receives executions (fills) from the matching engine and builds the order books and candlestick charts from the stream of executions



Reporting Flow

- Not on the critical trading flow
- It provides trading history, tax reporting, compliance reporting, settlements, etc
- Reporter is less sensitive to latency. Accuracy and compliance are key factors for the reporter
- An incoming new order only contains order details, and outgoing execution usually only contains order ID, price, quantity, and execution status. The reporter merges the attributes from both sources for the reports



Data Models

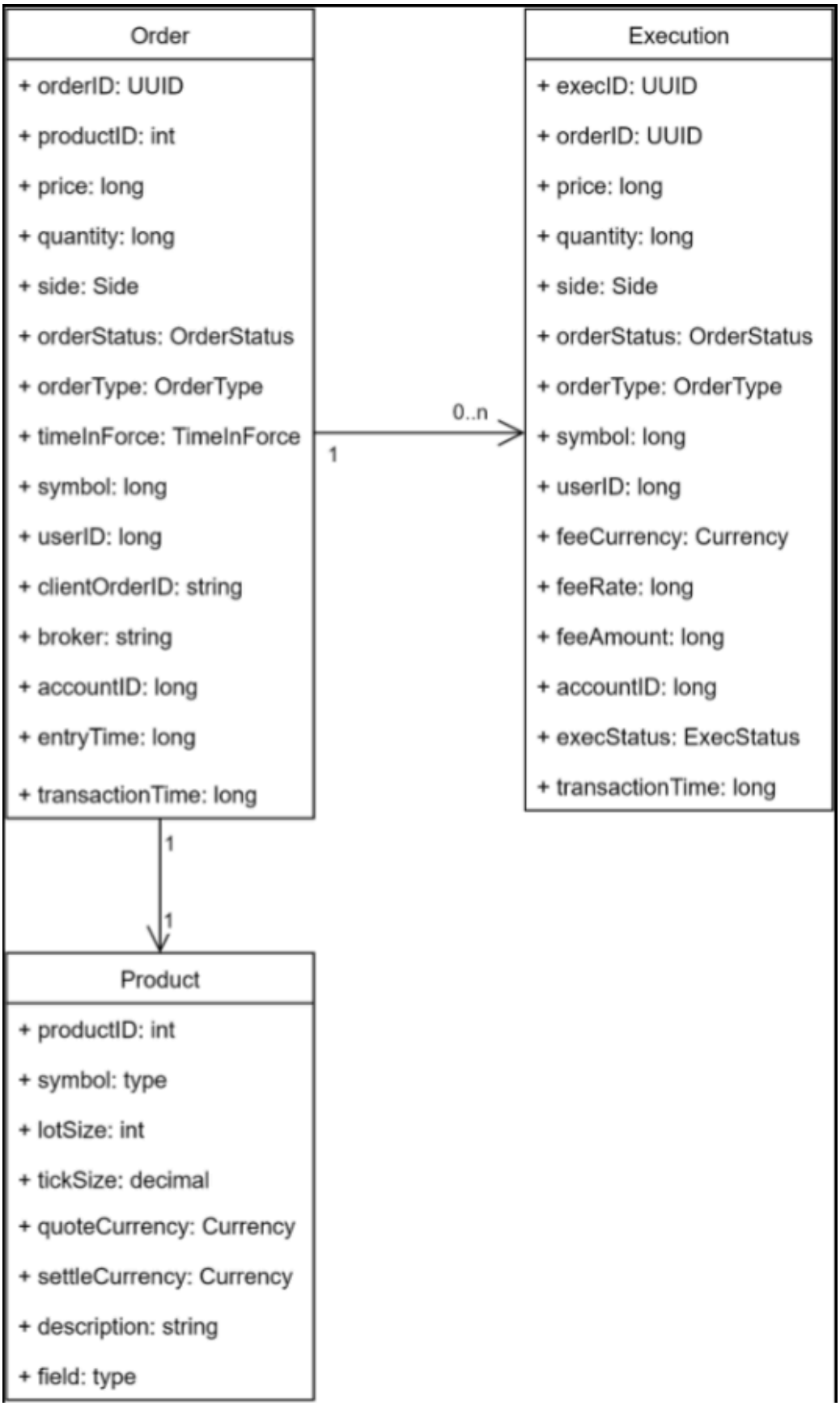
- Product, order & execution
- Order book
- Candlestick chart

Product, Order & Execution

- In the critical trading path, orders & executions are not stored in a DB because they are too slow. To achieve performance, this path executes trades in

memory & leverages hard disk or shared memory to persist & share orders & executions

- Orders & executions are stored in the sequencer for fast recovery & the data is archived after the market closes
- Reporter writes orders & executions to the DB for reporting use cases like reconciliation & tax reporting
- Executions are forwarded to the market data processor to reconstruct the order book & candlestick chart data

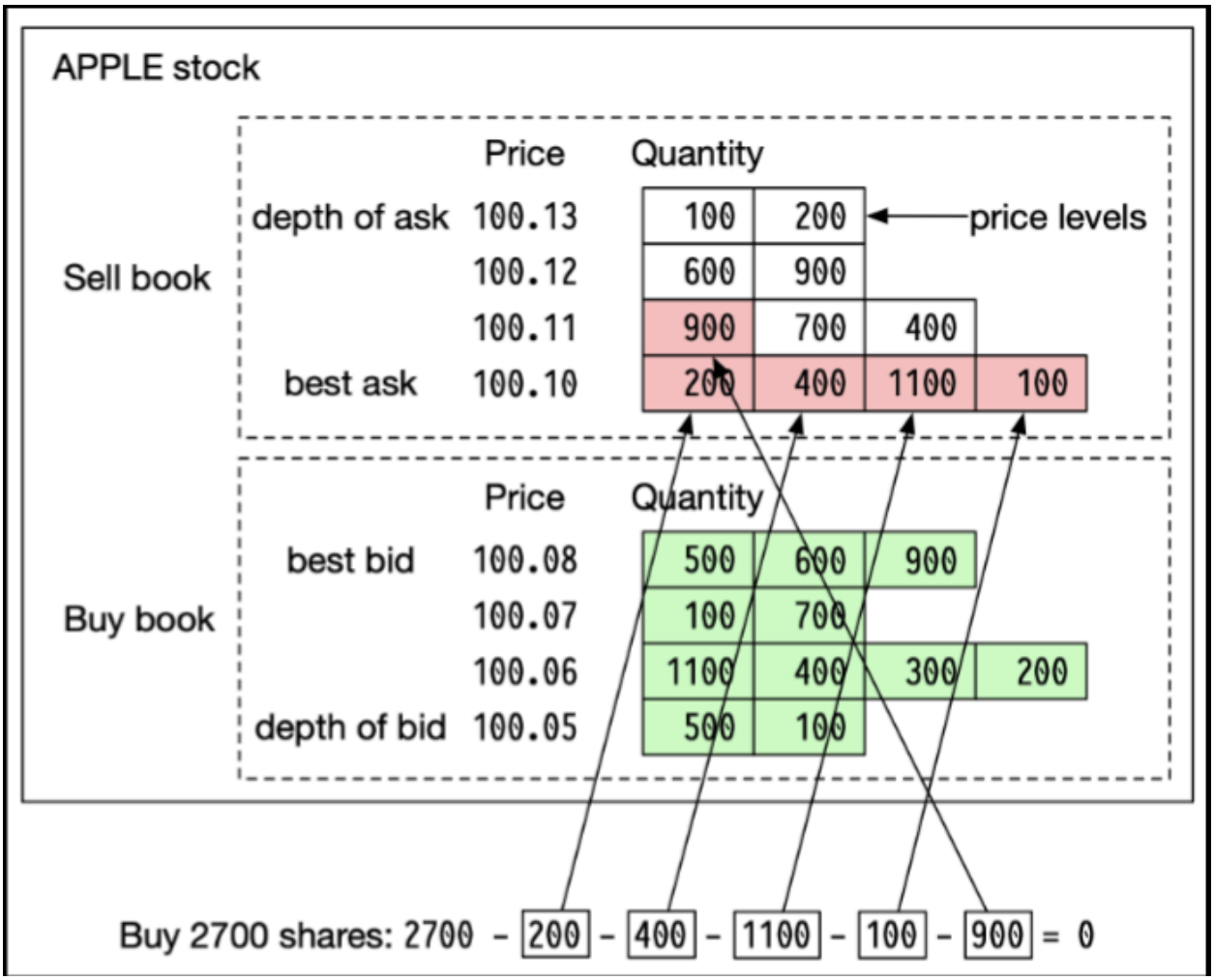

Order
+ orderID: UUID
+ productID: int
+ price: long
+ quantity: long
+ side: Side
+ orderStatus: OrderStatus
+ orderType: OrderType
+ timeInForce: TimeInForce
+ symbol: long
+ userID: long
+ clientOrderID: string
+ broker: string
+ accountID: long
+ entryTime: long
+ transactionTime: long
Execution
+ execID: UUID
+ orderID: UUID
+ price: long
+ quantity: long
+ side: Side
+ orderStatus: OrderStatus
+ orderType: OrderType
+ symbol: long
+ userID: long
+ feeCurrency: Currency
+ feeRate: long
+ feeAmount: long
+ accountID: long
+ execStatus: ExecStatus
+ transactionTime: long
0..n
1
1
1
Product
+ productID: int
+ symbol: type
+ lotSize: int
+ tickSize: decimal
+ quoteCurrency: Currency
+ settleCurrency: Currency
+ description: string
+ field: type

Order Book

- A list of buy & sell orders for a specific security or financial instrument, organized by price level
- Constant lookup time — operation includes getting volume at a price level or between price levels
- Fast add / cancel / execute operations, preferably O(1) time complexity — operations include placing a new order, canceling an order & matching an order
- Fast update — operations include replacing an order
- Query best bid / ask
- Iterate through price levels

What real operations look like ?

- Placing a new order means adding a new order to the tail of the priceLevel
- Matching an order means deleting an order from the head of the priceLevel
- Canceling an order means deleting an order from the orderBook — we leverage the helper data structure `Map<OrderID, Order>` orderMap in the orderBook to find the order to be cancelled and remove the order from the priceLevel — this deletion operation is O(1) time complexity for a doubly linked list



Candlestick chart

- Use preallocated ring buffers to hold sticks to reduce the number of new object allocations
- Limit the number of sticks in the memory & persist the rest to disk

Performance Optimizations

- Latency must be low and stable
- There are 2 mechanisms to reduce latency : -
  - Decrease the number of tasks in the critical path
  - Shorten the time spent on each task

    - by reducing or eliminating network & disk usage
    - by reducing execution time for each task
- Optimized critical path only contains the following components, even logging is removed

$$gateway \rightarrow order\ manager \rightarrow sequencer \rightarrow matching\ engine$$

- These money milking mofos have gotten to a point where network hops and disk calls seem like bottlenecks, they are literally jamming every critical path component onto the same server and using MMAPs and collocation to bring down latency ... yeah, READ THAT SENTENCE AGAIN — and here I thought 200ms SLA was tough

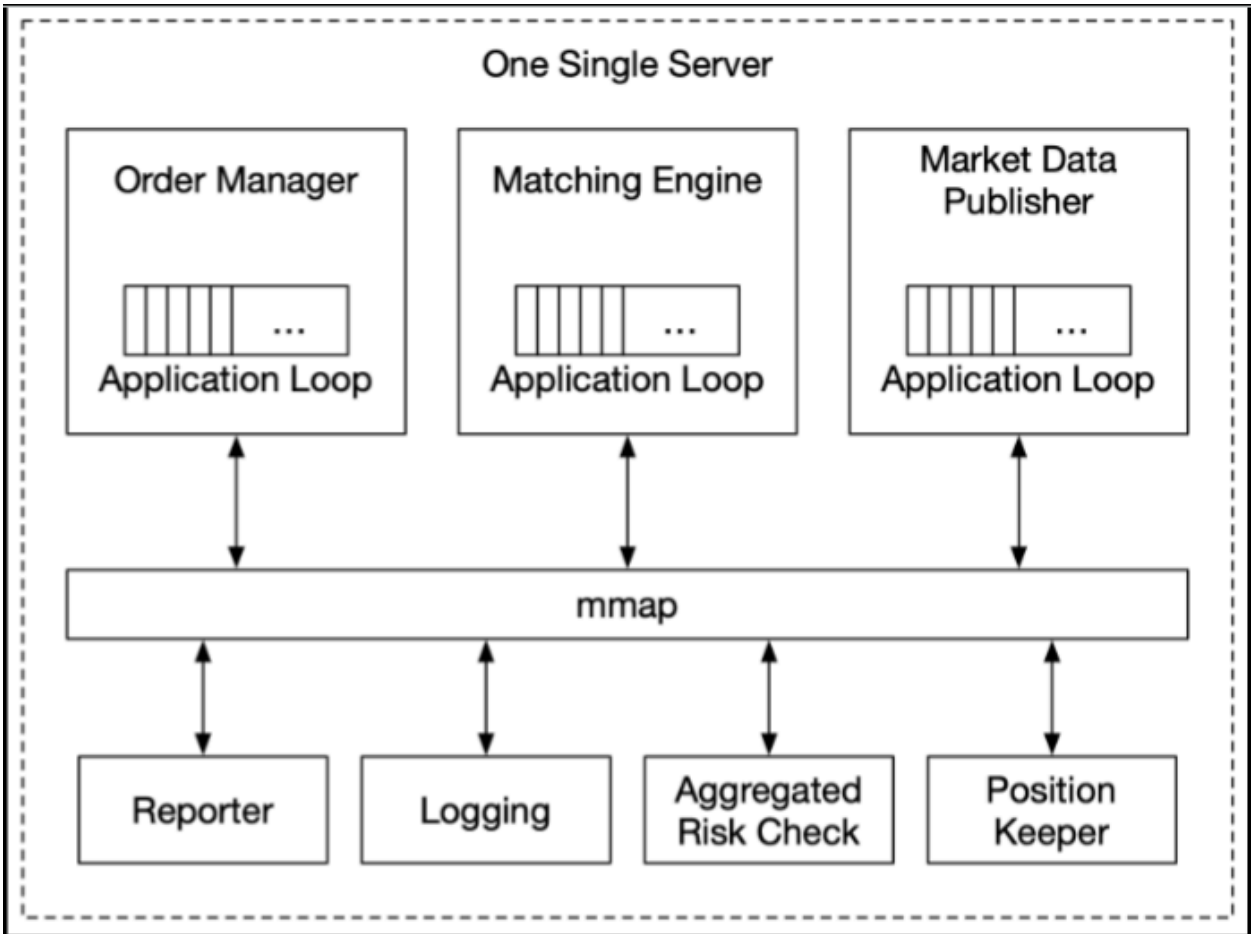
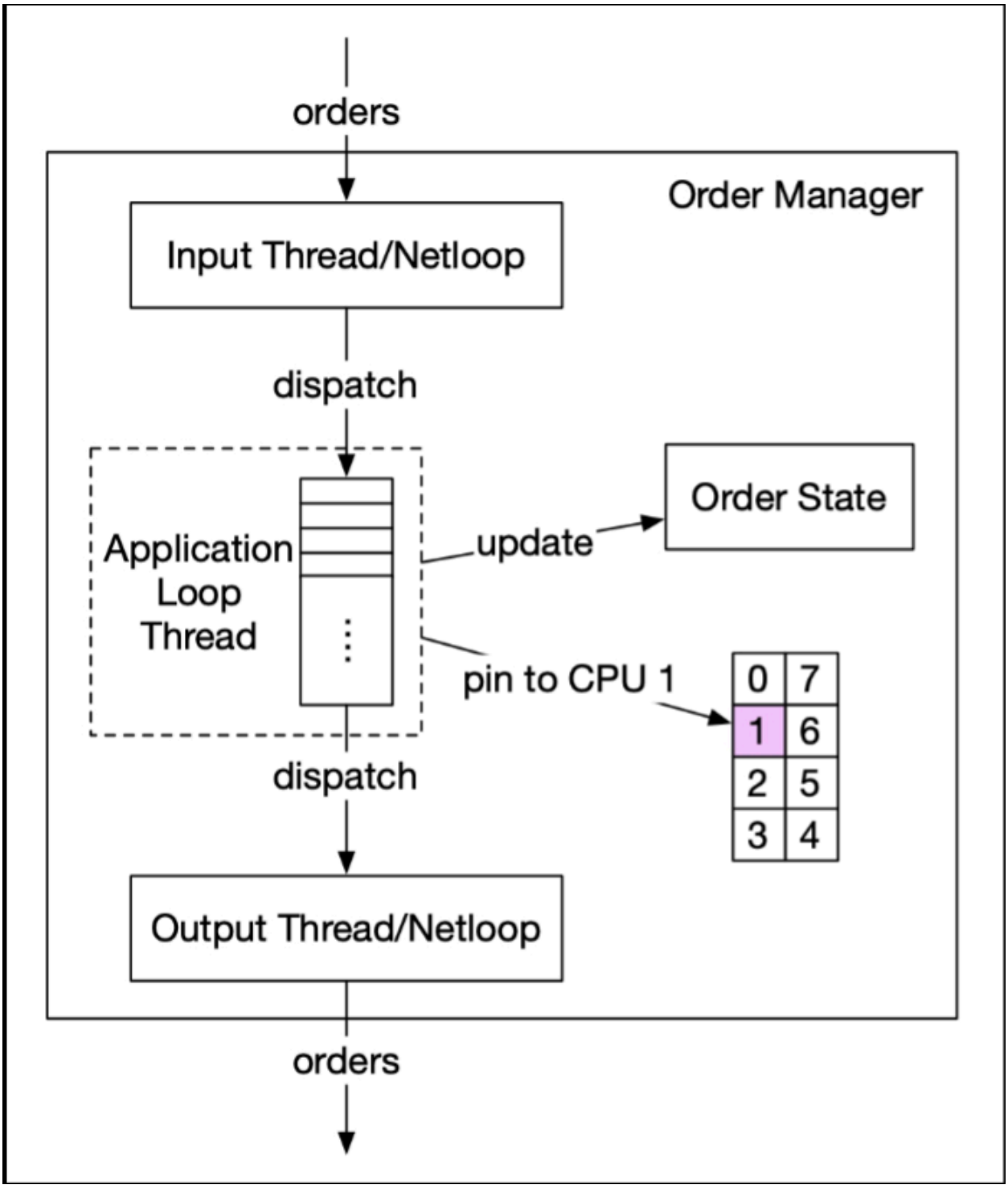


Figure 15 A low latency single server exchange design

Application loops

- It keeps polling for tasks to execute in a while loop and is the primary task execution mechanism
- To meet the strict latency budget, only the most mission-critical tasks should be processed by the application loop
- Its goal is to reduce the execution time for each component and to guarantee a highly predictable execution time (i.e., a low 99th percentile latency)
- To maximize CPU efficiency, each application loop (think of it as the main processing loop) is single-threaded, and the thread is pinned to a fixed CPU core — cache locality, code locality, low context switching latency & less cache coherence traffic overhead & no locks which means no lock contention

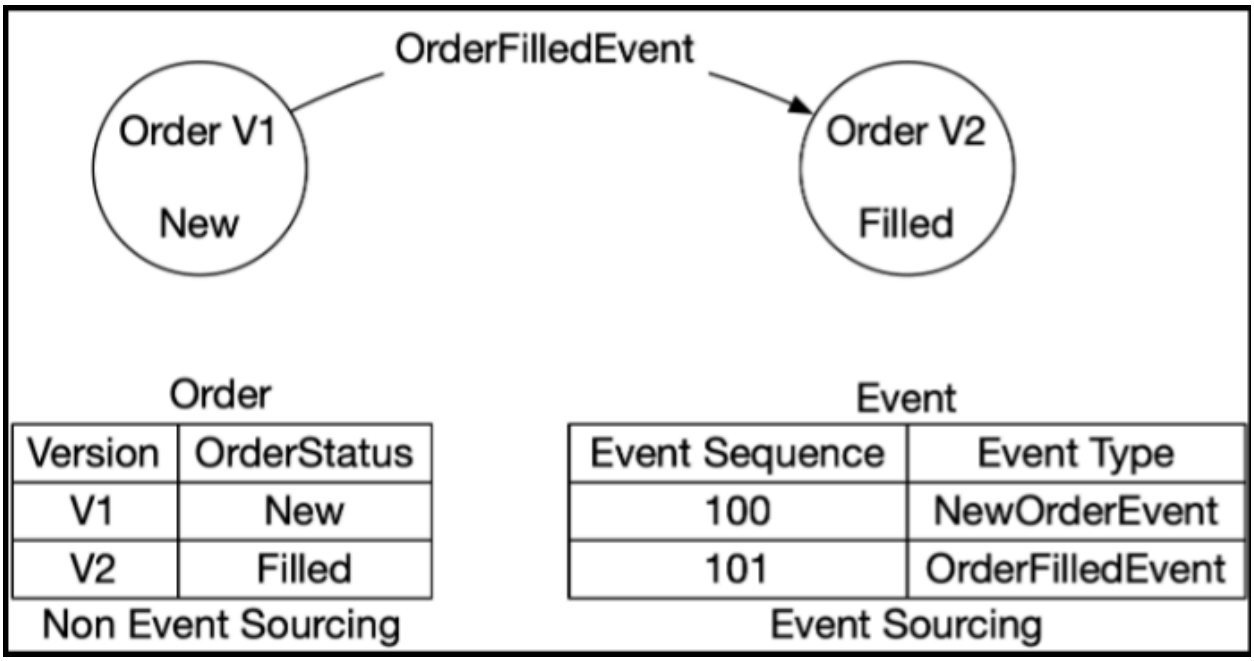


- MMAP is POSIX-compliant UNIX system call that maps a file into the memory of a process
- The performance advantage is compounded when the backing file is in `/dev/shm`

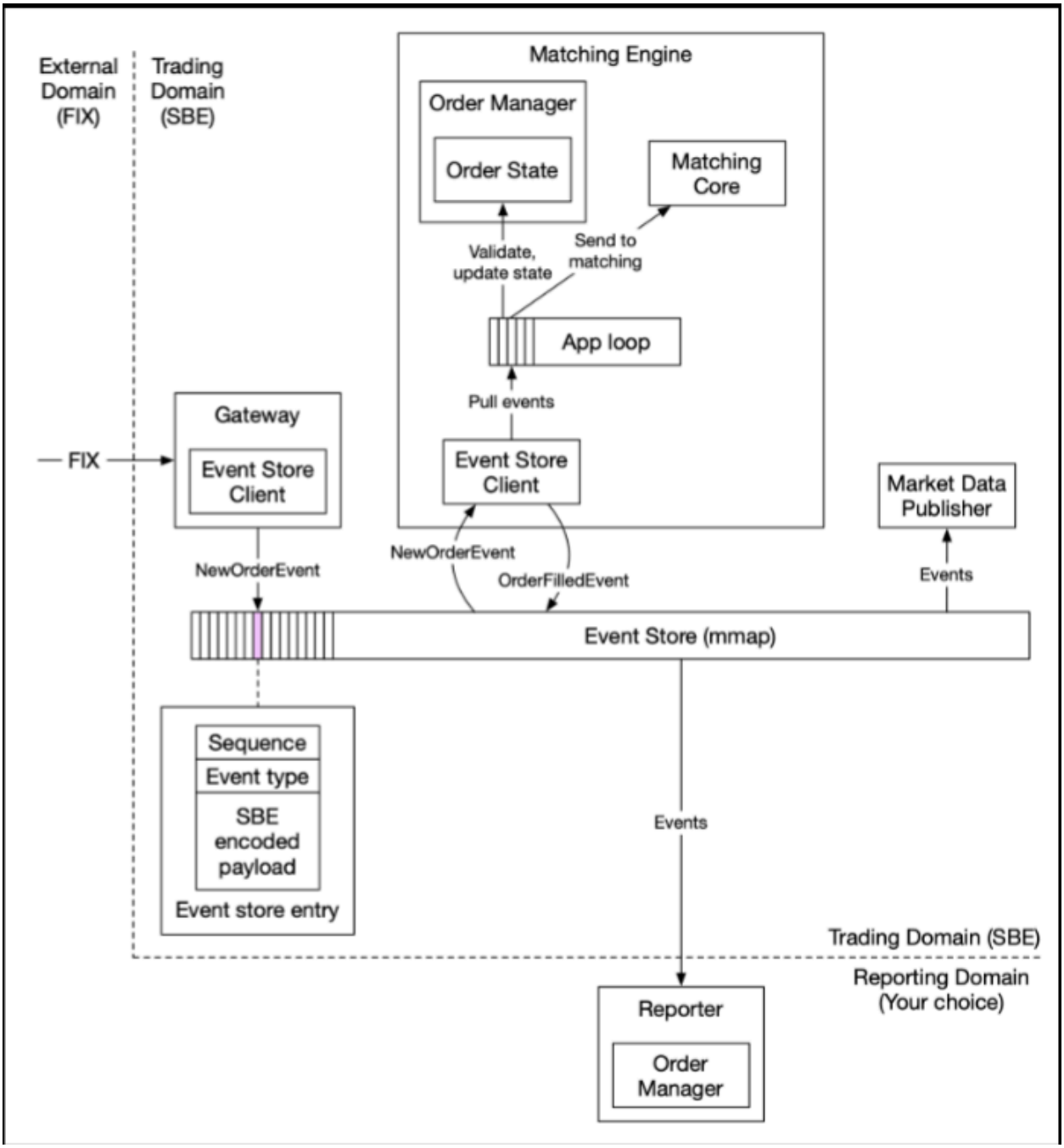
- `/dev/shm` is a memory-backed file system. When MMAP is done over a file in `/dev/shm`, the access to the shared memory does not result in any disk access at all
- MMAP is used in the server to implement a message bus over which the components on the critical path communicate. By leveraging MMAP to build an event store, coupled with the event sourcing design paradigm, modern exchanges can build low-latency microservices inside a server

Event Sourcing

- Discussed in depth in the Digital Wallet chapter
- In event sourcing, instead of storing the current states, it keeps an immutable log of all state-changing events — these events are the golden source of truth



- Refer this diagram to understand the flow in case of a stock exchange system



- If the latency requirements are not strict then the MMAP layer can be replaced with Kafka
- Gateway transforms FIX to “FIX over Simple Binary Encoding (SBE)” for fast & compact encoding & sends each order as a NewOrderEvent via the Event Store Client in a predefined format

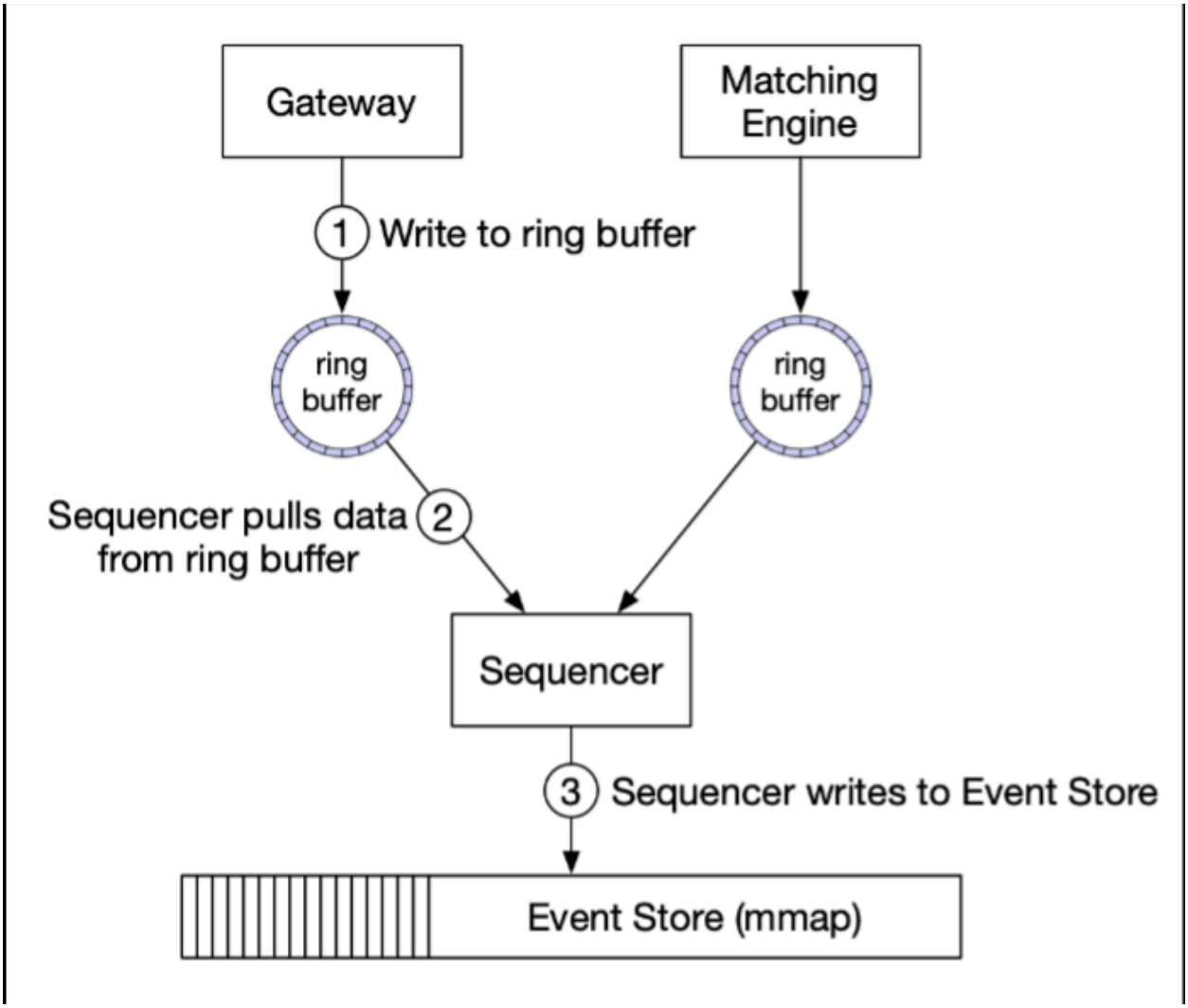
- Order Manager receives the NewOrderEvent from the event store, validates it, and adds it to its internal order states. The order is then sent to the matching core
- If the order gets matched, an OrderFilledEvent is generated & sent to the event store
- Other components like the market data processor & the reporter subscribe to the event store & process those events

Event Store paradigm

Requires certain adjustments to our architecture

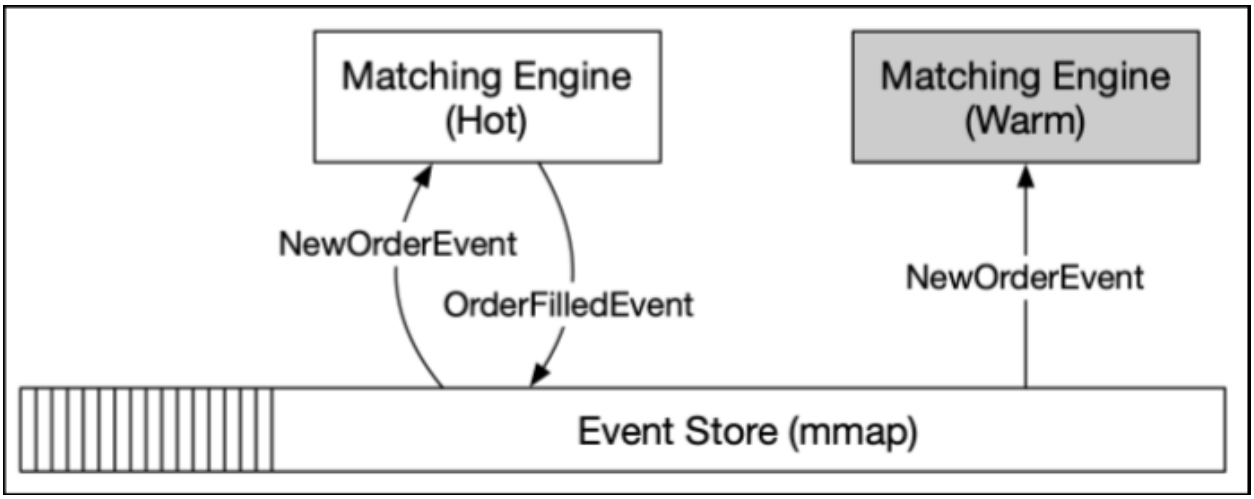
- The order manager becomes a reusable library that is embedded in different components because states of the orders are important for multiple components
- Having a centralized order manager for other components to update or query the order states would hurt latency, especially if those components are not on the critical trading path
- Although each component maintains the order states by itself, with event sourcing the states are guaranteed to be identical and replayable
- There is only one sequencer for each event store. It is bad practice to have multiple sequencers, as they will fight for the right to write to the event store
- In a system like an exchange, a lot of time would be wasted on lock contention. Therefore, the sequencer is a single writer which sequences the events before sending them to the event store
- Unlike the sequencer in the high-level design which also functions as a message store, the sequencer here only does one simple thing and is super fast. The sequencer pulls events from the ring buffer that is local to each component. For each event, it stamps a sequence ID on the event and sends it to the event store
- We can have backup sequencer for availability in case the primary sequencer goes down

Simple sequencer design



High Availability

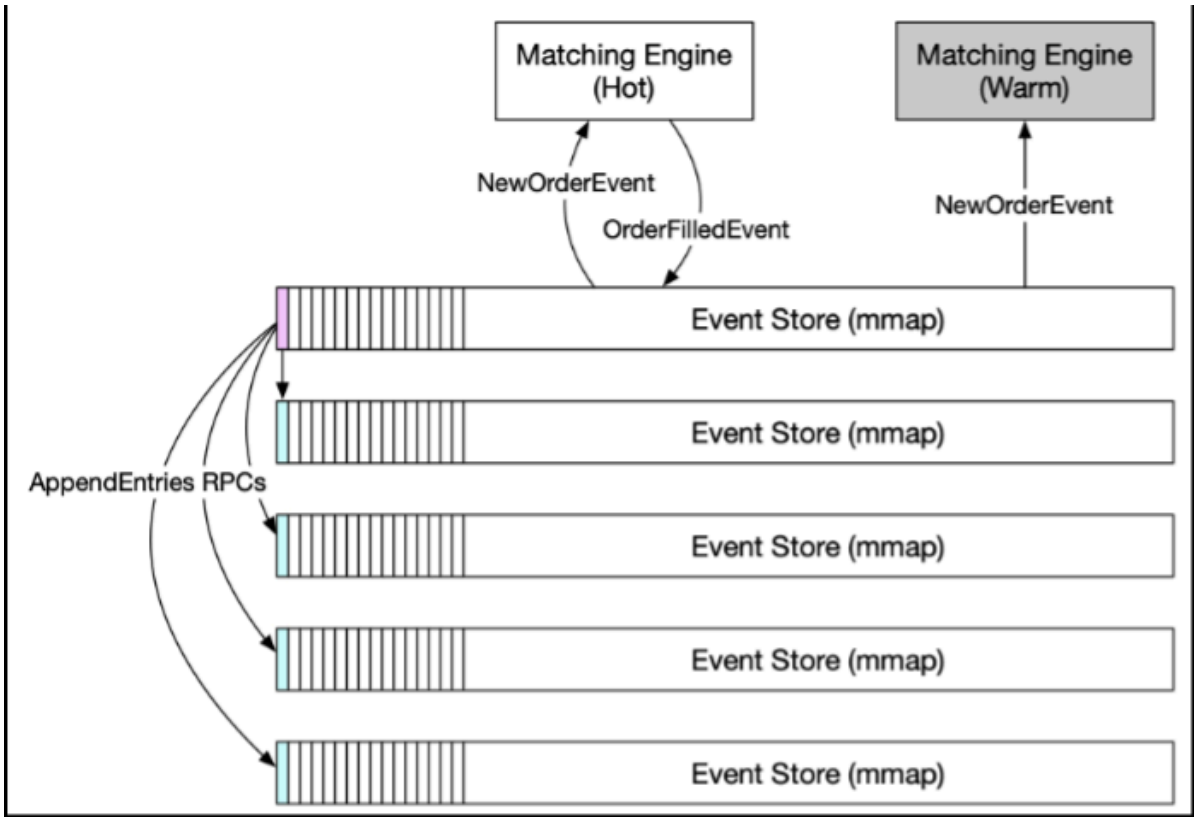
- Identify single points of failure & set up redundant instances alongside the primary instance
- Detection of failure and the decision to fail-over to the backup instance should be fast
- Primary candidates for high availability optimizations are : -
  - Client Gateway — stateless service so it can be scaled horizontally
  - Matching Engine & Order Manager — stateful component and must follow hot-warm paradigm for instantaneous fail-over



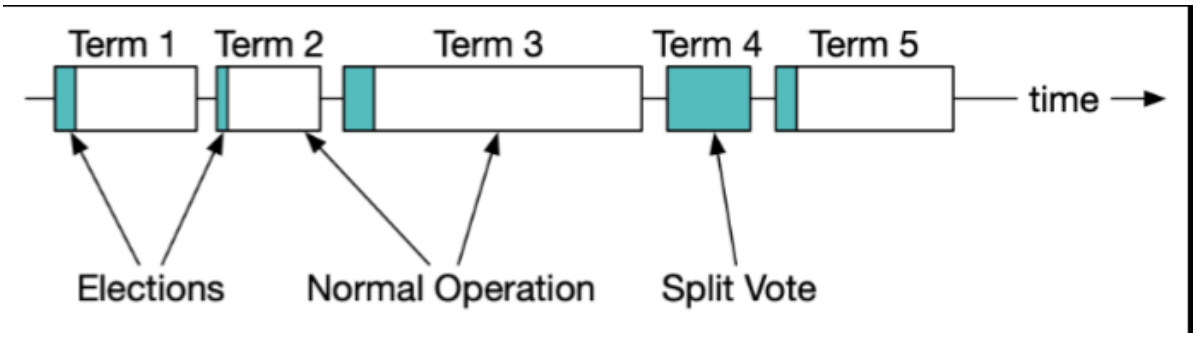
- The hot matching engine works as the primary instance, and the warm engine receives and processes the exact same events but does not send any event out onto the event store
- When the warm secondary instance goes down, upon restart, it can always recover all the states from the event store. Event sourcing is a great fit for the exchange architecture. The inherent determinism makes state recovery easy and accurate

Fault Tolerance

- We need to use RAFT consensus



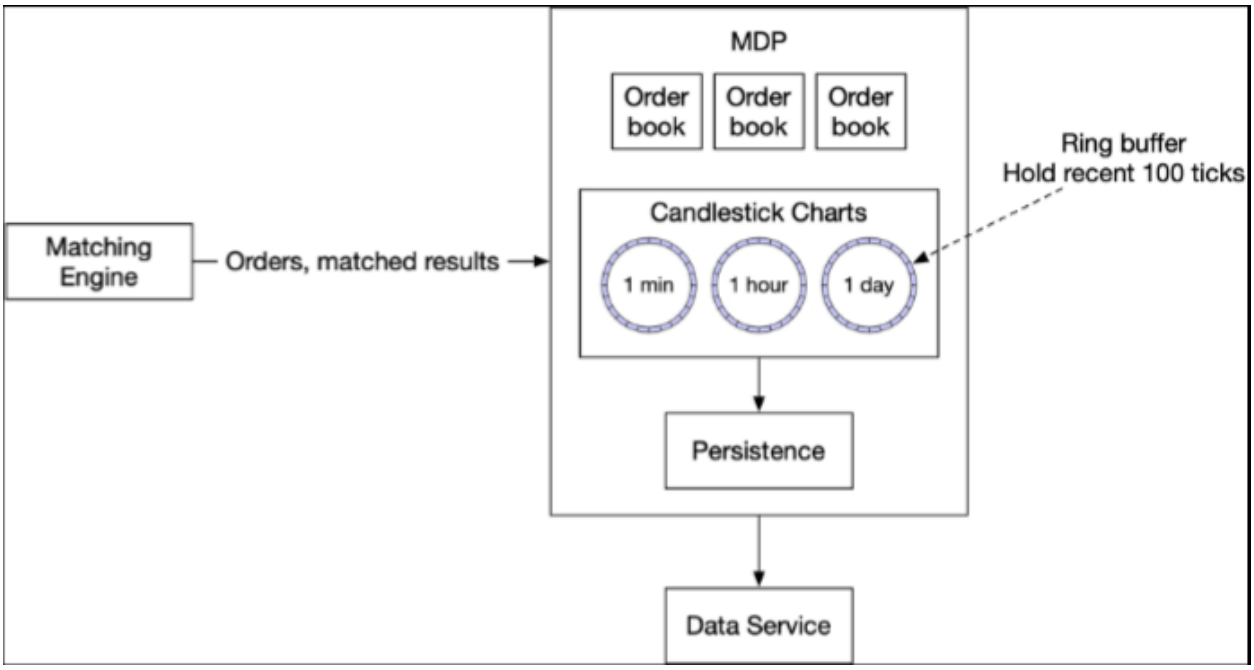
- The leader sends heartbeat messages (AppendEntries with no content) to its followers.
- If a follower has not received heartbeat messages for a period of time, it triggers an election timeout that initiates a new election.
- The first follower that reaches election timeout becomes a candidate, and it asks the rest of the followers to vote (RequestVote). If the first follower receives a majority of votes, it becomes the new leader.
- If the first follower has a lower term (WTF is a term — refer diagram below) value than the new node, it cannot be the leader. If multiple followers become candidates at the same time, it is called a “split vote”. In this case, the election times out, and a new election is initiated.



- Time is divided into arbitrary intervals in Raft to represent normal operation and election
- Recovery Point Objective (RPO) refers to the amount of data that can be lost before significant harm is done to the business, i.e. the loss tolerance

Market Data Publisher Optimizations

- The market data publisher (MDP) receives matched results from the matching engine and rebuilds the order book and candlestick charts based on that. It then publishes the data to the subscribers
- This design utilizes ring buffers which is a fixed size queue with the head connected to the tail. Producer continuously produces data and consumer continuously pulls data off it. Space in a ring buffer is preallocated so object creation & deallocation is unnecessary. The data structure is also lock free
- Distribution fairness is crucial and can be achieved using following mechanisms : -
    - Multicast + reliable UDP : all receivers in the same multicast group will receive the datagram at the same time
    - Colocation : paid for VIP service which allows hedge funds & brokers to put their servers in the same data structure as the exchange

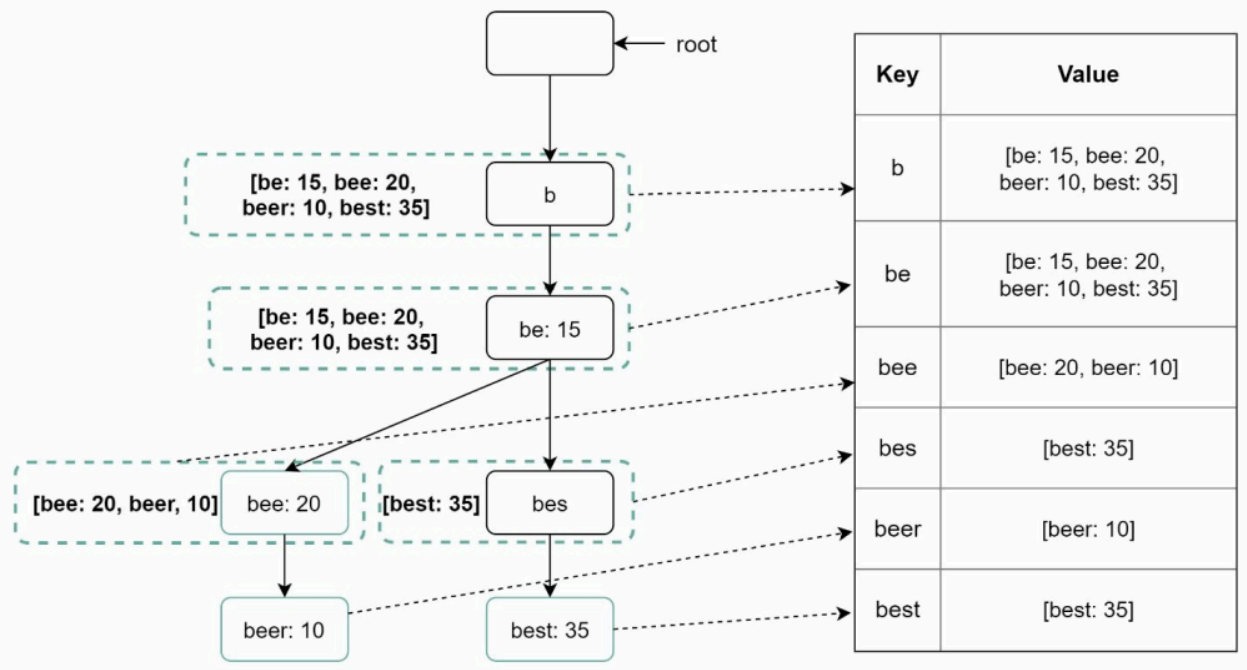


Search Autocomplete (top-k)

Design revolves around 3 major components : -

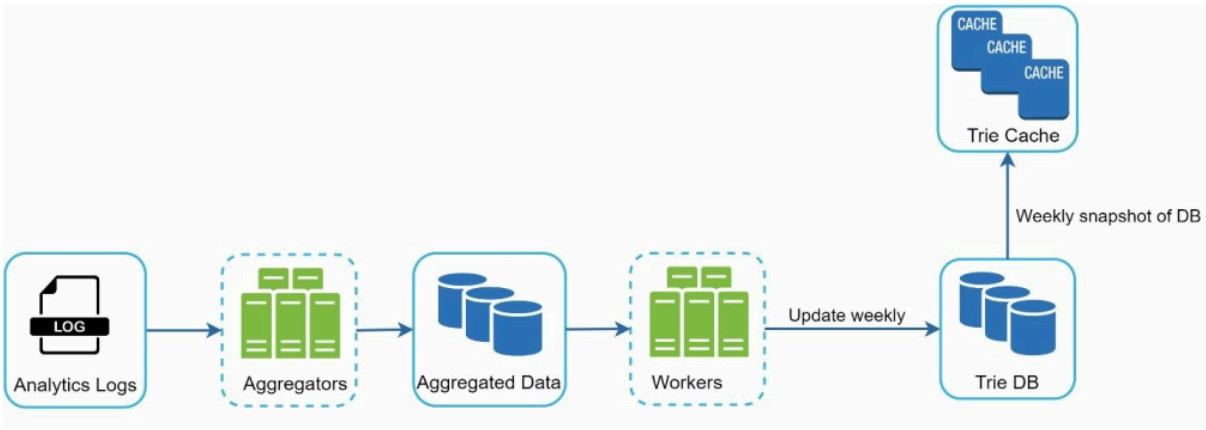
Trie data structure

- To avoid traversing the whole trie, we store top k most frequently used queries at each node
  - Find the prefix node — time complexity: O(1)
  - Return top k — since top k queries are cached, the time complexity for this step is O(1)
- These tries get huge at scale so apart from the cached keys, rest of the data is kept in key-value store
  - Every prefix in the trie is mapped to a key in a hash table
  - Data on each trie node is mapped to a value in a hash table

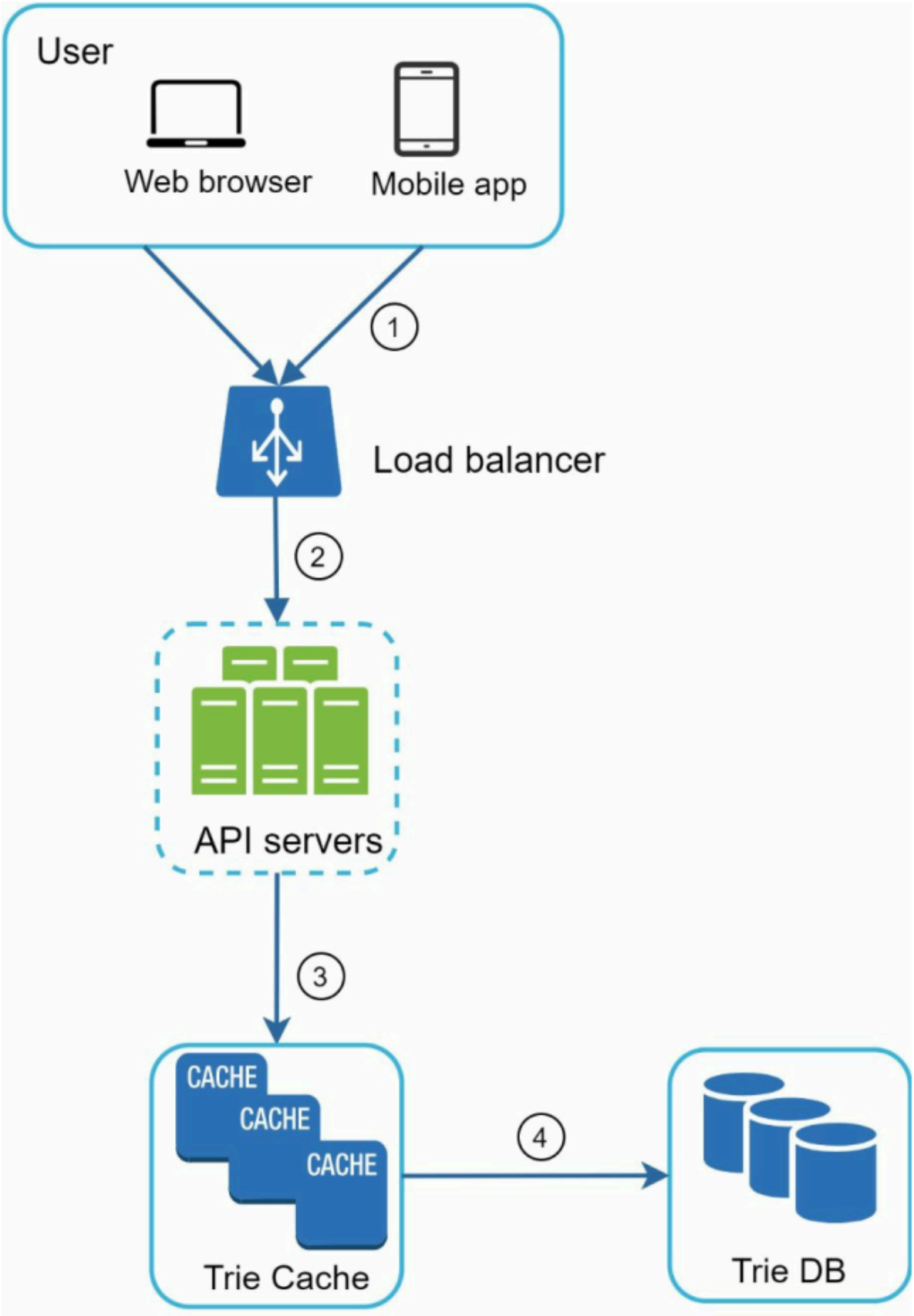


Data gathering service

- Usually built from analytics or logging service



Query service


User
Web browser
Mobile app
1
Load balancer
2
API servers
3
CACHE
CACHE
CACHE
Trie Cache
4
Trie DB

Honestly, this is a pretty standard problem ... just master prefix lists and tries ... Hail Mary